

□ DeepGen-Protein: Representation Learning and Generative Modeling of Protein Sequences

MSc Artificial Intelligence — Goa University Team: Kowsya Mutkundu | Lisa Luis | Muskan Khatoon | Shannon Coelho

Colab Setup: Upload the `random_split` folder directly to Colab's file system (left panel). It should be visible at `/content/random_split/` with `dev/`, `test/`, `train/` inside. Set Runtime → Change runtime type → **GPU (T4)**.

```
# — CELL 1: Imports & Setup —
import os, random, math, time, glob
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (accuracy_score, f1_score,
                             confusion_matrix,
                             classification_report)
from sklearn.manifold import TSNE

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.benchmark = True
torch.backends.cudnn.deterministic = False

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')
if DEVICE.type == 'cuda':
    print(f'GPU: {torch.cuda.get_device_name(0)}')
    print(f'Memory: {torch.cuda.get_device_properties(0).total_memory
/ 1e9:.1f} GB')
```

Using device: cuda
GPU: Tesla T4
Memory: 15.6 GB

```
# — CELL 2: Load Dataset —————  
DATA_DIR = '/content/random_split/'  
  
def load_split(folder):  
    fs = sorted(glob.glob(folder + 'data-*'))  
    print(f'{os.path.basename(folder.rstrip("/"))}: {len(fs)} shards')  
    dfs = []  
    for f in fs:  
        dfs.append(pd.read_csv(f))  
    return pd.concat(dfs, ignore_index=True)  
  
train_df = load_split(DATA_DIR + 'train/')  
dev_df    = load_split(DATA_DIR + 'dev/')  
test_df   = load_split(DATA_DIR + 'test/')  
  
print(f'\nTrain: {len(train_df):,} | Dev: {len(dev_df):,} | Test:  
{len(test_df):,}')  
print(f'Columns: {train_df.columns.tolist()}')  
train_df.head()  
  
train: 80 shards  
dev: 10 shards  
test: 10 shards  
  
Train: 1,086,741 | Dev: 126,171 | Test: 126,171  
Columns: ['family_id', 'sequence_name', 'family_accession',  
'aligned_sequence', 'sequence']  
  
{"type": "dataframe", "variable_name": "train_df"}  
  
# — CELL 3: Use Top-20 Families + Label Encoding —————  
TOP_FAMILIES = 20  
top_fam =  
train_df['family_id'].value_counts().head(TOP_FAMILIES).index  
  
train_df =  
train_df[train_df['family_id'].isin(top_fam)].reset_index(drop=True)  
dev_df    =  
dev_df[dev_df['family_id'].isin(top_fam)].reset_index(drop=True)  
test_df   =  
test_df[test_df['family_id'].isin(top_fam)].reset_index(drop=True)  
  
print(f'After filtering to top {TOP_FAMILIES} families:')  
print(f'Train: {len(train_df):,} | Dev: {len(dev_df):,} | Test:  
{len(test_df):,}')  
  
label_enc = LabelEncoder()
```

```

label_enc.fit(train_df['family_id'])
NUM_CLASSES = len(label_enc.classes_)

train_df['label'] = label_enc.transform(train_df['family_id'])
dev_df['label'] = label_enc.transform(dev_df['family_id'])
test_df['label'] = label_enc.transform(test_df['family_id'])

print(f'Number of classes: {NUM_CLASSES}')
print(f'Families: {label_enc.classes_.tolist()}')

After filtering to top 20 families:
Train: 28,179 | Dev: 3,511 | Test: 3,511
Number of classes: 20
Families: ['Acetyltransf_7', 'Bac_transf', 'Chromate_transp', 'DHH',
'DNA_binding_1', 'DnaJ_CXXCXGXG', 'Glycos_trans_3N', 'HTH_1',
'His_kinase', 'LRR_1', 'Lipase_GDSL_2', 'Lum_binding', 'Maf',
'Methyltransf_25', 'OTCace_N', 'PEP-utilizers', 'Prenyltransf',
'SRP54_N', 'THF_DHG_CYH', 'WD40']

# — CELL 4: EDA Plots —————
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

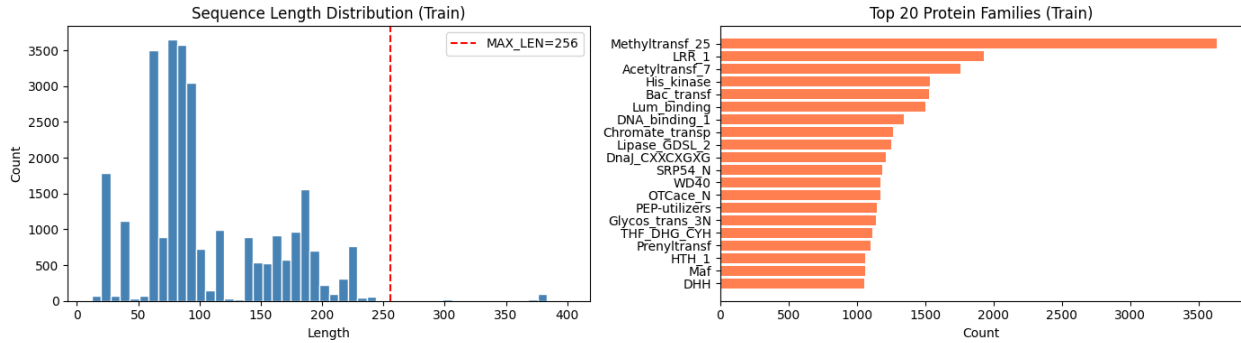
seq_lens = train_df['sequence'].apply(len)
axes[0].hist(seq_lens.clip(upper=1000), bins=50, color='steelblue',
edgecolor='white')
axes[0].axvline(256, color='red', linestyle='--', label='MAX_LEN=256')
axes[0].set_title('Sequence Length Distribution (Train)')
axes[0].set_xlabel('Length'); axes[0].set_ylabel('Count')
axes[0].legend()

top20 = train_df['family_id'].value_counts().head(20)
axes[1].barh(top20.index[::-1], top20.values[::-1], color='coral')
axes[1].set_title('Top 20 Protein Families (Train)')
axes[1].set_xlabel('Count')

plt.tight_layout()
plt.savefig('eda_plots.png', dpi=150, bbox_inches='tight')
plt.show()

print(f'Mean seq length: {seq_lens.mean():.1f} | Median:
{seq_lens.median():.1f}')

```



Mean seq length: 106.6 | Median: 87.0

— CELL 5: Amino Acid Encoding & Dataset —

```
AMINO_ACIDS = list('ACDEFGHIKLMNPQRSTVWY')
PAD_TOKEN   = '<PAD>'
UNK_TOKEN    = '<UNK>'
vocab       = [PAD_TOKEN] + AMINO_ACIDS + [UNK_TOKEN]
aa2idx      = {aa: i for i, aa in enumerate(vocab)}
idx2aa      = {i: aa for aa, i in aa2idx.items()}
VOCAB_SIZE  = len(vocab)
MAX_LEN     = 256
BATCH_SIZE  = 128 # Safe for P100

def encode_sequence(seq, max_len=MAX_LEN):
    seq = seq.upper()[:max_len]
    encoded = [aa2idx.get(aa, aa2idx[UNK_TOKEN]) for aa in seq]
    padded = encoded + [0] * (max_len - len(encoded))
    return torch.tensor(padded, dtype=torch.long)

class ProteinDataset(Dataset):
    def __init__(self, df):
        self.sequences = df['sequence'].values
        self.labels     = df['label'].values
    def __len__(self): return len(self.sequences)
    def __getitem__(self, idx):
        return encode_sequence(self.sequences[idx]),
        torch.tensor(self.labels[idx], dtype=torch.long)

train_loader = DataLoader(ProteinDataset(train_df),
    batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
dev_loader   = DataLoader(ProteinDataset(dev_df),
    batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)
test_loader  = DataLoader(ProteinDataset(test_df),
    batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)

print(f'Train batches: {len(train_loader)} | Dev: {len(dev_loader)} |
Test: {len(test_loader)}')
```

Train batches: 221 | Dev: 28 | Test: 28

```

# — CELL 6: Training Utilities —————
def train_one_epoch(model, loader, optimizer, criterion, scaler):
    model.train()
    total_loss, correct, total = 0, 0, 0
    for x, y in tqdm(loader, leave=False):
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        with torch.cuda.amp.autocast():
            logits = model(x)
            loss = criterion(logits, y)
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()
        total_loss += loss.item() * y.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        total += y.size(0)
    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0, 0, 0
    all_preds, all_labels = [], []
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        logits = model(x)
        loss = criterion(logits, y)
        total_loss += loss.item() * y.size(0)
        preds = logits.argmax(1)
        correct += (preds == y).sum().item()
        total += y.size(0)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(y.cpu().numpy())
    f1 = f1_score(all_labels, all_preds, average='macro',
zero_division=0)
    return total_loss / total, correct / total, f1, all_preds,
all_labels

def plot_curves(history, title):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    ep = range(1, len(history['train_loss']) + 1)
    axes[0].plot(ep, history['train_loss'], label='Train')
    axes[0].plot(ep, history['val_loss'], label='Val')
    axes[0].set_title(f'{title} - Loss'); axes[0].legend()
    axes[1].plot(ep, history['train_acc'], label='Train')
    axes[1].plot(ep, history['val_acc'], label='Val')
    axes[1].set_title(f'{title} - Accuracy'); axes[1].legend()

```

```

plt.tight_layout()
plt.savefig(f'{title}_curves.png', dpi=150)
plt.show()

def train_model(model, train_loader, dev_loader, epochs=10, lr=1e-3,
model_name='model', patience=3):
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=epochs)
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
    scaler = torch.cuda.amp.GradScaler()
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [],
'val_acc': []}
    best_val_acc, best_state = 0.0, None
    counter = 0

    for epoch in range(1, epochs + 1):
        t0 = time.time()
        tr_loss, tr_acc = train_one_epoch(model, train_loader,
optimizer, criterion, scaler)
        vl_loss, vl_acc, vl_f1, _, _ = evaluate(model, dev_loader,
criterion)
        scheduler.step()

        history['train_loss'].append(tr_loss)
        history['val_loss'].append(vl_loss)
        history['train_acc'].append(tr_acc)
        history['val_acc'].append(vl_acc)

        if vl_acc > best_val_acc:
            best_val_acc = vl_acc
            best_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()}
            counter = 0
        else:
            counter += 1

        print(f'Epoch {epoch:02d}/{epochs} | Train Loss: {tr_loss:.4f}
Acc: {tr_acc:.4f} | '
            f'Val Loss: {vl_loss:.4f} Acc: {vl_acc:.4f} F1:
{vl_f1:.4f} | Time: {time.time()-t0:.1f}s')

        if counter >= patience:
            print(f'Early stopping triggered at epoch {epoch}')
            break

    model.load_state_dict({k: v.to(DEVICE) for k, v in
best_state.items()})
    torch.save(model.state_dict(), f'{model_name}_best.pt')

```

```

        plot_curves(history, model_name)
        return history

print('Training utilities ready!')

Training utilities ready!

# — CELL 7: Model 1 – 1D CNN —————
class CNN1DClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes,
dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.conv_block1 = nn.Sequential(
            nn.Conv1d(embed_dim, 128, kernel_size=3, padding=1),
            nn.BatchNorm1d(128), nn.ReLU(), nn.MaxPool1d(2))
        self.conv_block2 = nn.Sequential(
            nn.Conv1d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm1d(256), nn.ReLU(), nn.MaxPool1d(2))
        self.conv_block3 = nn.Sequential(
            nn.Conv1d(256, 512, kernel_size=3, padding=1),
            nn.BatchNorm1d(512), nn.ReLU(), nn.MaxPool1d(2))
        self.global_pool = nn.AdaptiveAvgPool1d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512, 256), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(256, num_classes))

    def forward(self, x):
        e = self.embedding(x).permute(0, 2, 1)
        e = self.conv_block1(e)
        e = self.conv_block2(e)
        e = self.conv_block3(e)
        return self.classifier(self.global_pool(e))

cnn_model = CNN1DClassifier(VOCAB_SIZE, embed_dim=64,
num_classes=NUM_CLASSES).to(DEVICE)
print(f'CNN params: {sum(p.numel() for p in cnn_model.parameters() if
p.requires_grad):,}')

print('='*60)
print('Training 1D CNN...')
print('='*60)
cnn_history = train_model(cnn_model, train_loader, dev_loader,
epochs=12, model_name='CNN1D', patience=3)

CNN params: 656,660
=====

```

Training 1D CNN...

```
=====
/tmp/ipykernel_5168/4233851654.py:56: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler()
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():
```

Epoch 01/12 | Train Loss: 1.1431 Acc: 0.8458 | Val Loss: 0.7018 Acc: 0.9977 F1: 0.9975 | Time: 8.0s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():
```

Epoch 02/12 | Train Loss: 0.6786 Acc: 0.9988 | Val Loss: 0.6484 Acc: 0.9997 F1: 0.9996 | Time: 3.1s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():
```

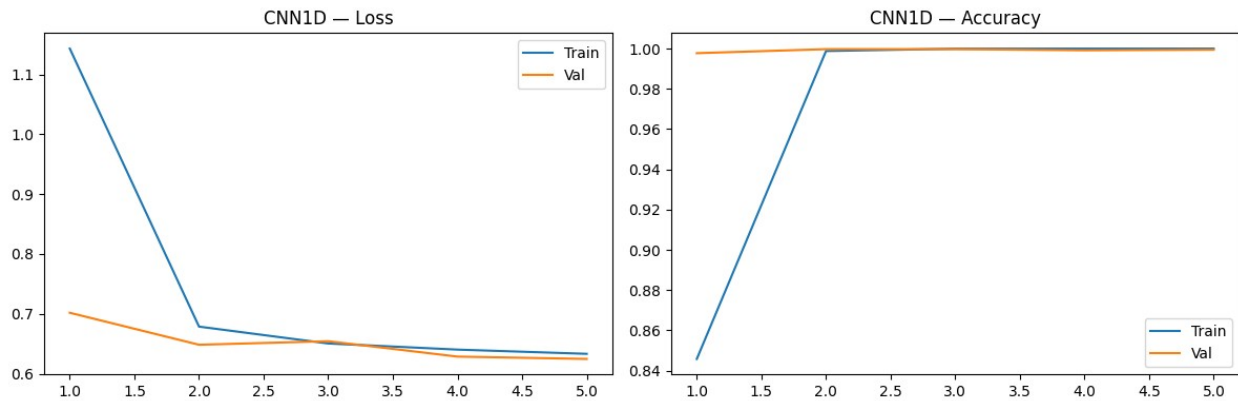
Epoch 03/12 | Train Loss: 0.6505 Acc: 0.9999 | Val Loss: 0.6543 Acc: 0.9997 F1: 0.9996 | Time: 3.1s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():
```

Epoch 04/12 | Train Loss: 0.6402 Acc: 1.0000 | Val Loss: 0.6286 Acc: 0.9991 F1: 0.9992 | Time: 4.3s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():
```


Epoch 05/12 | Train Loss: 0.6332 Acc: 1.0000 | Val Loss: 0.6247 Acc: 0.9994 F1: 0.9994 | Time: 3.1s
Early stopping triggered at epoch 5



```
# — CELL 8: Model 2 — LSTM —————
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes,
num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.lstm      = nn.LSTM(embed_dim, hidden_dim,
num_layers=num_layers,
                                batch_first=True,
bidirectional=True,
                                dropout=dropout if num_layers > 1
else 0)
        self.attn      = nn.Linear(hidden_dim * 2, 1)
        self.classifier= nn.Sequential(
            nn.Linear(hidden_dim * 2, 256), nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_classes))

    def forward(self, x):
        e  = self.embedding(x)
        out, _ = self.lstm(e)
        w  = torch.softmax(self.attn(out), dim=1)
        ctx = (out * w).sum(dim=1)
        return self.classifier(ctx)

lstm_model = LSTMClassifier(VOCAB_SIZE, embed_dim=64, hidden_dim=256,
num_classes=NUM_CLASSES).to(DEVICE)
print(f'LSTM params: {sum(p.numel() for p in lstm_model.parameters()
if p.requires_grad):,}')

print('='*60)
```

```

print('Training LSTM...')
print('='*60)
lstm_history = train_model(lstm_model, train_loader, dev_loader,
epochs=10, model_name='LSTM', patience=3)

/tmp/ipykernel_5168/4233851654.py:56: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler()

LSTM params: 2,374,805
=====
Training LSTM...
=====

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 01/10 | Train Loss: 2.0225 Acc: 0.4067 | Val Loss: 1.2042 Acc:
0.8015 F1: 0.7465 | Time: 14.7s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 02/10 | Train Loss: 1.0069 Acc: 0.8803 | Val Loss: 0.6887 Acc:
0.9863 F1: 0.9849 | Time: 13.6s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 03/10 | Train Loss: 0.6893 Acc: 0.9894 | Val Loss: 0.6232 Acc:
0.9952 F1: 0.9948 | Time: 13.8s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 04/10 | Train Loss: 0.6330 Acc: 0.9991 | Val Loss: 0.6050 Acc:
0.9997 F1: 0.9996 | Time: 14.6s

```

```

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast():

Epoch 05/10 | Train Loss: 0.6214 Acc: 0.9999 | Val Loss: 0.6033 Acc:
0.9983 F1: 0.9981 | Time: 14.4s

```

```

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast():

Epoch 06/10 | Train Loss: 0.6185 Acc: 1.0000 | Val Loss: 0.6002 Acc:
0.9994 F1: 0.9993 | Time: 14.6s

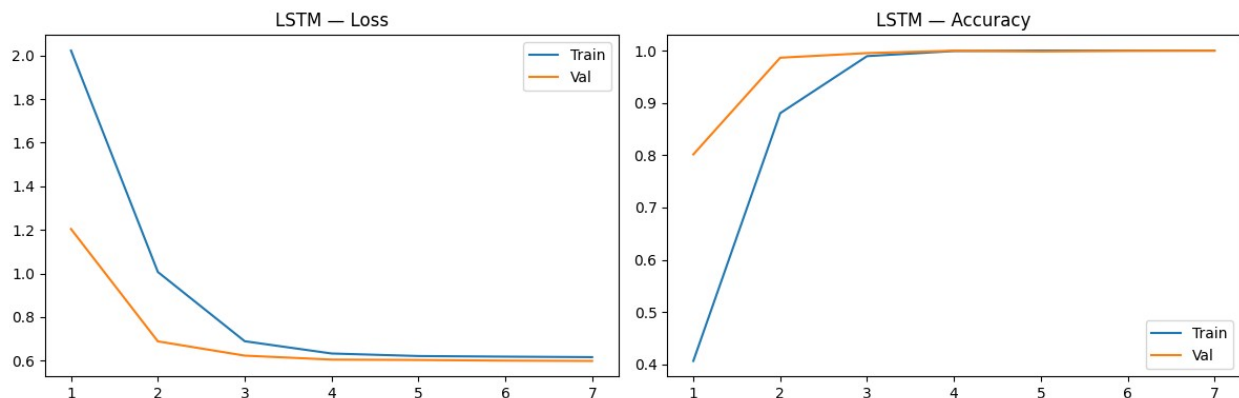
```

```

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast():

Epoch 07/10 | Train Loss: 0.6162 Acc: 1.0000 | Val Loss: 0.5988 Acc:
0.9997 F1: 0.9996 | Time: 14.5s
Early stopping triggered at epoch 7

```



```

# — CELL 9: Model 3 — GRU —————
class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes,
num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim,
num_layers=num_layers,

```

```

        batch_first=True, bidirectional=True,
        dropout=dropout if num_layers > 1
else 0)
    self.attn = nn.Linear(hidden_dim * 2, 1)
    self.classifier= nn.Sequential(
        nn.Linear(hidden_dim * 2, 256), nn.ReLU(),
nn.Dropout(dropout),
        nn.Linear(256, num_classes))

    def forward(self, x):
        e = self.embedding(x)
        out, _ = self.gru(e)
        w = torch.softmax(self.attn(out), dim=1)
        ctx = (out * w).sum(dim=1)
        return self.classifier(ctx)

gru_model = GRUClassifier(VOCAB_SIZE, embed_dim=64, hidden_dim=256,
num_classes=NUM_CLASSES).to(DEVICE)
print(f'GRU params: {sum(p.numel() for p in gru_model.parameters() if
p.requires_grad):,}')

print('='*60)
print('Training GRU...')
print('='*60)
gru_history = train_model(gru_model, train_loader, dev_loader,
epochs=10, model_name='GRU', patience=3)

/tmp/ipykernel_5168/4233851654.py:56: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler()

GRU params: 1,815,701
=====
Training GRU...
=====

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 01/10 | Train Loss: 1.4917 Acc: 0.6705 | Val Loss: 0.6571 Acc:
0.9934 F1: 0.9927 | Time: 12.6s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

```

Epoch 02/10 | Train Loss: 0.6494 Acc: 0.9980 | Val Loss: 0.6135 Acc: 0.9986 F1: 0.9985 | Time: 12.6s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():
```

Epoch 03/10 | Train Loss: 0.6239 Acc: 0.9999 | Val Loss: 0.6029 Acc: 0.9989 F1: 0.9988 | Time: 12.5s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():
```

Epoch 04/10 | Train Loss: 0.6164 Acc: 1.0000 | Val Loss: 0.6002 Acc: 0.9997 F1: 0.9996 | Time: 12.6s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():
```

Epoch 05/10 | Train Loss: 0.6133 Acc: 1.0000 | Val Loss: 0.5992 Acc: 0.9994 F1: 0.9994 | Time: 12.7s

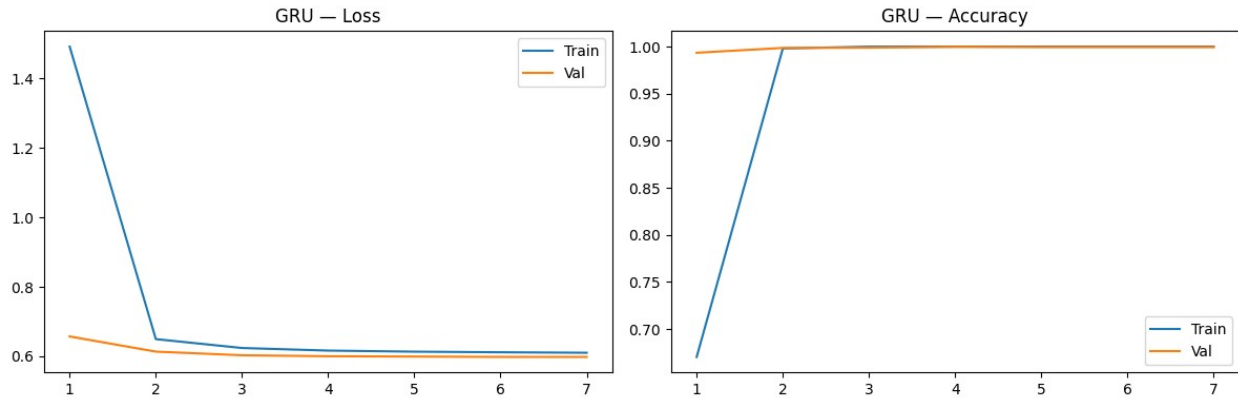
```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():
```

Epoch 06/10 | Train Loss: 0.6116 Acc: 1.0000 | Val Loss: 0.5981 Acc: 0.9994 F1: 0.9994 | Time: 12.6s

```
0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():
```

Epoch 07/10 | Train Loss: 0.6102 Acc: 1.0000 | Val Loss: 0.5981 Acc: 0.9994 F1: 0.9994 | Time: 12.7s

Early stopping triggered at epoch 7



```
# — CELL 9: Model 3 — GRU —————
class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes,
num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim,
num_layers=num_layers,
batch_first=True, bidirectional=True,
dropout=dropout if num_layers > 1
else 0)
        self.attn = nn.Linear(hidden_dim * 2, 1)
        self.classifier= nn.Sequential(
            nn.Linear(hidden_dim * 2, 256), nn.ReLU(),
nn.Dropout(dropout),
            nn.Linear(256, num_classes))

    def forward(self, x):
        e = self.embedding(x)
        out, _ = self.gru(e)
        w = torch.softmax(self.attn(out), dim=1)
        ctx = (out * w).sum(dim=1)
        return self.classifier(ctx)

    def get_representation(self, x):
        e = self.embedding(x)
        out, _ = self.gru(e)
        w = torch.softmax(self.attn(out), dim=1)
        return (out * w).sum(dim=1)

gru_model = GRUClassifier(VOCAB_SIZE, embed_dim=64, hidden_dim=256,
num_classes=NUM_CLASSES).to(DEVICE)
print(f'GRU params: {sum(p.numel() for p in gru_model.parameters() if
p.requires_grad):,}')

print('='*60)
```

```

print('Training GRU...')
print('='*60)
gru_history = train_model(gru_model, train_loader, dev_loader,
epochs=10, model_name='GRU')

/tmp/ipykernel_5168/4233851654.py:56: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler()

GRU params: 1,815,701
=====
Training GRU...
=====

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 01/10 | Train Loss: 1.5242 Acc: 0.6485 | Val Loss: 0.6620 Acc:
0.9929 F1: 0.9924 | Time: 12.7s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 02/10 | Train Loss: 0.6535 Acc: 0.9971 | Val Loss: 0.6128 Acc:
0.9994 F1: 0.9993 | Time: 12.7s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 03/10 | Train Loss: 0.6252 Acc: 0.9996 | Val Loss: 0.6049 Acc:
0.9991 F1: 0.9990 | Time: 12.8s

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

Epoch 04/10 | Train Loss: 0.6172 Acc: 1.0000 | Val Loss: 0.6011 Acc:
0.9997 F1: 0.9996 | Time: 12.7s

```

```

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():

Epoch 05/10 | Train Loss: 0.6137 Acc: 1.0000 | Val Loss: 0.5994 Acc:
0.9994 F1: 0.9993 | Time: 12.7s

```

```

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():

Epoch 06/10 | Train Loss: 0.6117 Acc: 1.0000 | Val Loss: 0.5998 Acc:
0.9991 F1: 0.9990 | Time: 12.8s

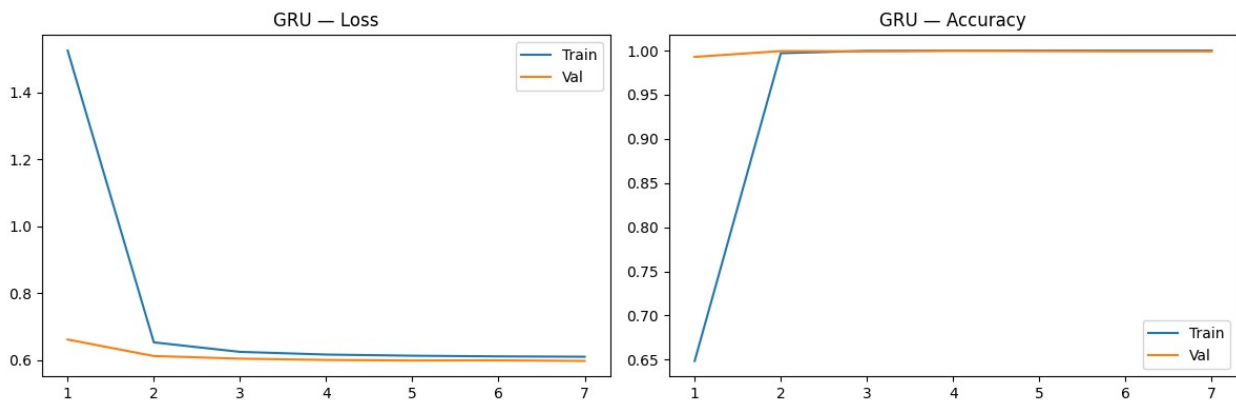
```

```

0%|          | 0/221 [00:00<?,
?it/s]/tmp/ipykernel_5168/4233851654.py:8: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast():

Epoch 07/10 | Train Loss: 0.6106 Acc: 1.0000 | Val Loss: 0.5981 Acc:
0.9991 F1: 0.9990 | Time: 12.7s
Early stopping triggered at epoch 7

```



```

# — CELL 10: ISA-1 Evaluation on Test Set —————
criterion = nn.CrossEntropyLoss()
results = {}

print('Final Test Evaluation')
print('='*60)
for name, model in [('1D-CNN', cnn_model), ('LSTM', lstm_model),
('GRU', gru_model)]:
    _, acc, f1, preds, labels = evaluate(model, test_loader,

```



```

criterion)
    results[name] = {'accuracy': acc, 'f1_macro': f1, 'preds': preds,
                    'labels': labels}
    print(f'{name:10s} | Accuracy: {acc:.4f} | Macro-F1: {f1:.4f}')

# Bar chart
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
names = list(results.keys())
accs = [results[n]['accuracy'] for n in names]
f1s = [results[n]['f1_macro'] for n in names]
colors = ['steelblue', 'coral', 'seagreen']

axes[0].bar(names, accs, color=colors)
axes[0].set_ylim(0, 1); axes[0].set_title('Test Accuracy')
for i, v in enumerate(accs): axes[0].text(i, v+0.01, f'{v:.3f}',
ha='center')

axes[1].bar(names, f1s, color=colors)
axes[1].set_ylim(0, 1); axes[1].set_title('Macro-F1 Score')
for i, v in enumerate(f1s): axes[1].text(i, v+0.01, f'{v:.3f}',
ha='center')

plt.suptitle('ISA-1 – Model Comparison', fontsize=13,
fontweight='bold')
plt.tight_layout()
plt.savefig('model_comparison.png', dpi=150)
plt.show()

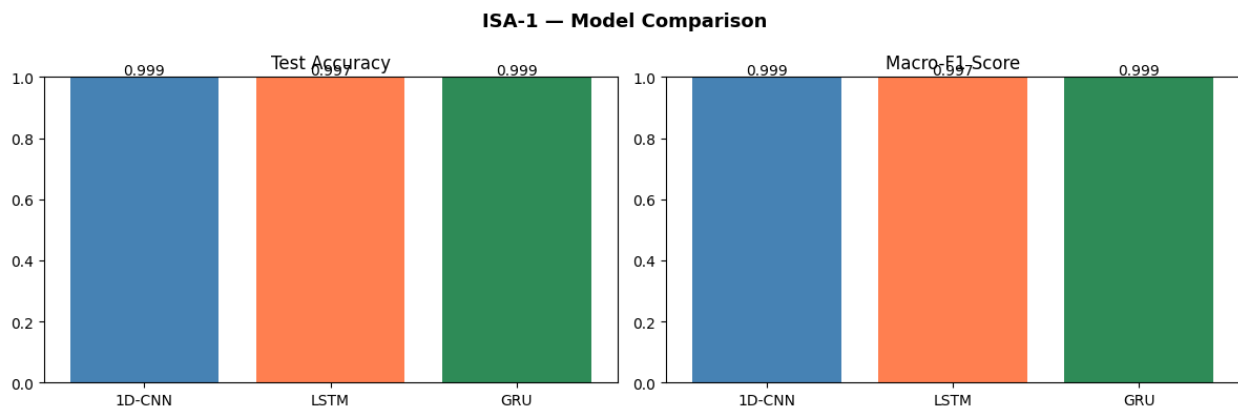
```

Final Test Evaluation

```

=====
1D-CNN | Accuracy: 0.9994 | Macro-F1: 0.9994
LSTM   | Accuracy: 0.9974 | Macro-F1: 0.9972
GRU    | Accuracy: 0.9994 | Macro-F1: 0.9993

```



```

# — CELL 11: Confusion Matrix —————
best_name = max(results, key=lambda k: results[k]['accuracy'])

```

```

print(f'Best model on test: {best_name}')

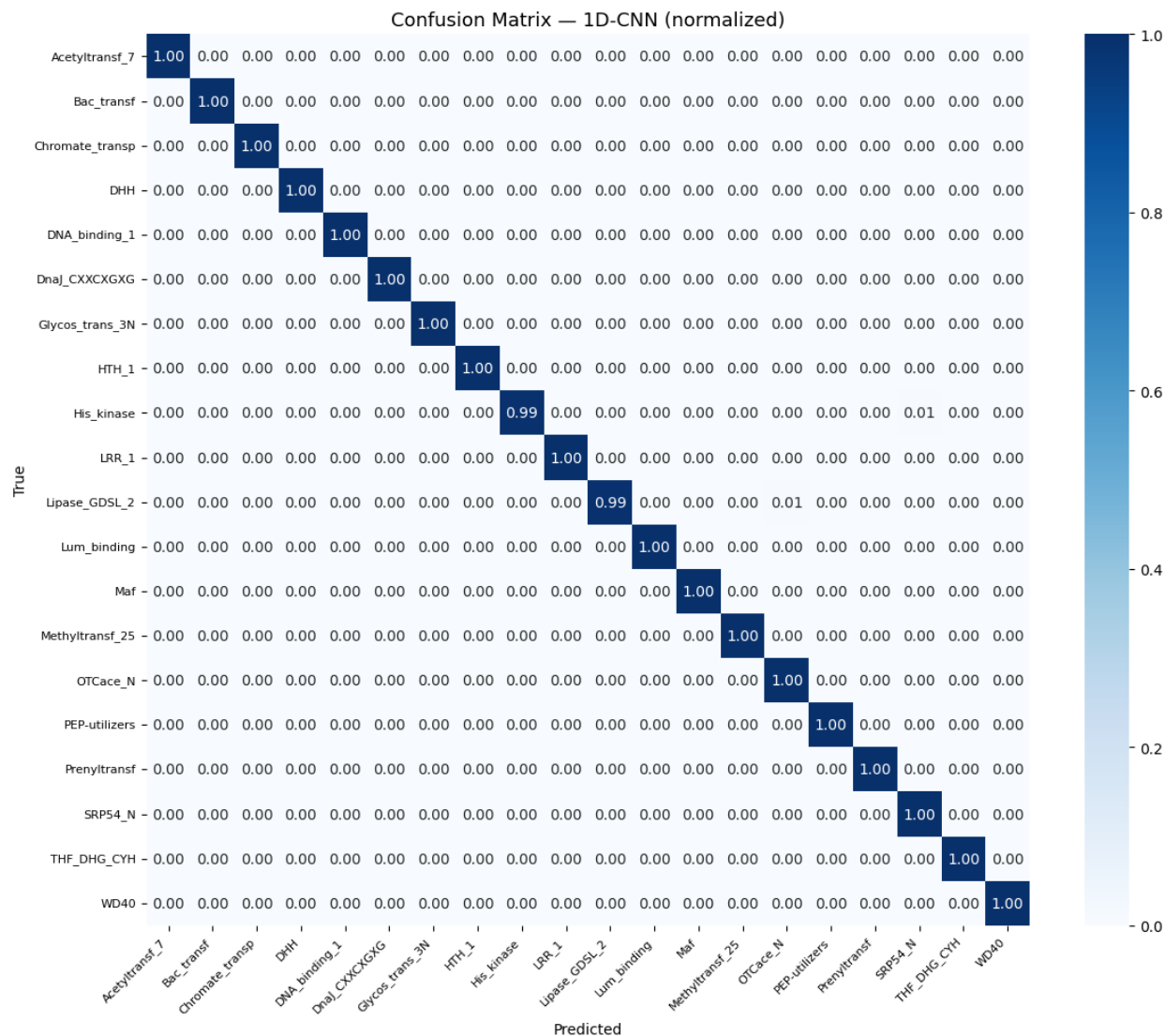
preds = results[best_name]['preds']
labels = results[best_name]['labels']
cm = confusion_matrix(labels, preds)
cm_norm = cm.astype('float') / (cm.sum(axis=1, keepdims=True) + 1e-9)

plt.figure(figsize=(12, 10))
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
            xticklabels=label_enc.classes_,
            yticklabels=label_enc.classes_)
plt.title(f'Confusion Matrix - {best_name} (normalized)', fontsize=13)
plt.xlabel('Predicted'); plt.ylabel('True')
plt.xticks(rotation=45, ha='right', fontsize=8)
plt.yticks(fontsize=8)
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()

print(classification_report(labels, preds,
                            target_names=label_enc.classes_, zero_division=0))

```

Best model on test: 1D-CNN

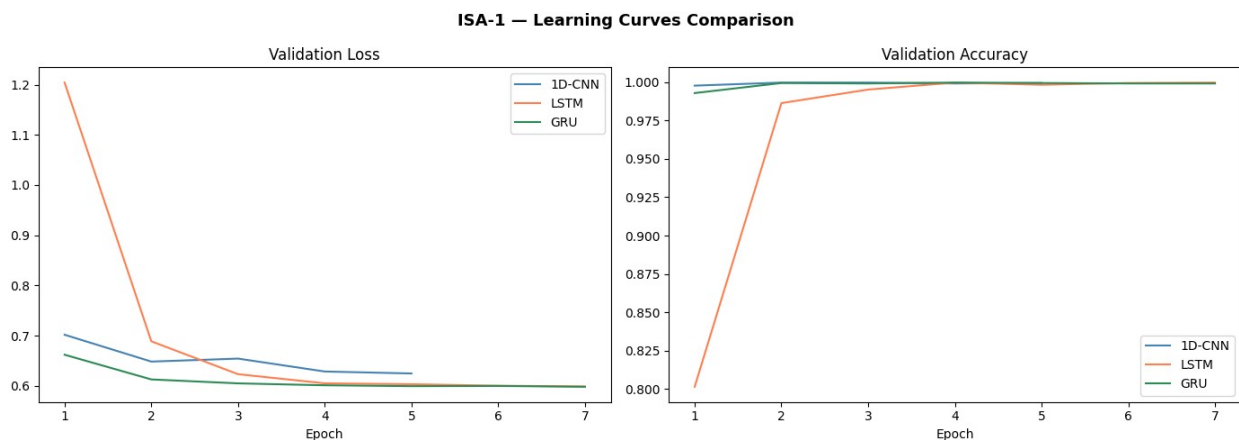


	precision	recall	f1-score	support
Acetyltransf_7	1.00	1.00	1.00	219
Bac_transf	1.00	1.00	1.00	190
Chromate_transp	1.00	1.00	1.00	157
DHH	1.00	1.00	1.00	131
DNA_binding_1	1.00	1.00	1.00	168
DnaJ_CXXCXGXG	1.00	1.00	1.00	151
Glycos_trans_3N	1.00	1.00	1.00	142
HTH_1	1.00	1.00	1.00	133
His_kinase	1.00	0.99	1.00	192
LRR_1	1.00	1.00	1.00	240
Lipase_GDSL_2	1.00	0.99	1.00	156
Lum_binding	1.00	1.00	1.00	187
Maf	1.00	1.00	1.00	132
Methyltransf_25	1.00	1.00	1.00	454

OTCace_N	0.99	1.00	1.00	146
PEP-utilizers	1.00	1.00	1.00	143
Prenyltransf	1.00	1.00	1.00	138
SRP54_N	0.99	1.00	1.00	148
THF_DHG_CYH	1.00	1.00	1.00	138
WD40	1.00	1.00	1.00	146
accuracy			1.00	3511
macro avg	1.00	1.00	1.00	3511
weighted avg	1.00	1.00	1.00	3511

```
# — CELL 12: Learning Curves Overlay —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for hist, name, color in [
    (cnn_history, '1D-CNN', 'steelblue'),
    (lstm_history, 'LSTM', 'coral'),
    (gru_history, 'GRU', 'seagreen')]:
    ep = range(1, len(hist['val_loss']) + 1)
    axes[0].plot(ep, hist['val_loss'], label=name, color=color)
    axes[1].plot(ep, hist['val_acc'], label=name, color=color)

axes[0].set_title('Validation Loss'); axes[0].set_xlabel('Epoch');
axes[0].legend()
axes[1].set_title('Validation Accuracy'); axes[1].set_xlabel('Epoch');
axes[1].legend()
plt.suptitle('ISA-1 – Learning Curves Comparison', fontsize=13,
fontweight='bold')
plt.tight_layout()
plt.savefig('learning_curves_all.png', dpi=150)
plt.show()
```



ISA-2: Generative Models

```
# — CELL 13: VAE Dataset —————
class ProteinSeqDataset(Dataset):
    def __init__(self, sequences):
        self.sequences = sequences
    def __len__(self): return len(self.sequences)
    def __getitem__(self, idx): return
        encode_sequence(self.sequences[idx])

vae_loader = DataLoader(
    ProteinSeqDataset(train_df['sequence'].values),
    batch_size=64, shuffle=True, num_workers=2, pin_memory=True) #
    smaller batch for VAE

LATENT_DIM = 128
print(f'VAE training sequences: {len(train_df):,} | Latent dim:
{LATENT_DIM}')

VAE training sequences: 28,179 | Latent dim: 128

# — CELL 14: VAE Architecture —————
class ProteinEncoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, latent_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers=2,
                        batch_first=True, bidirectional=True,
dropout=0.2)
        self.fc_mu = nn.Linear(hidden_dim * 2, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim * 2, latent_dim)

    def forward(self, x):
        e = self.embedding(x)
        _, h = self.gru(e)
        h = torch.cat([h[-2], h[-1]], dim=-1)
        return self.fc_mu(h), self.fc_logvar(h)

class ProteinDecoder(nn.Module):
    def __init__(self, latent_dim, hidden_dim, vocab_size, max_len,
embed_dim):
        super().__init__()
        self.max_len = max_len
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.fc_z = nn.Linear(latent_dim, hidden_dim)
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers=2,
batch_first=True, dropout=0.2)
```

```

        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, z, target_seq=None, teacher_forcing_ratio=0.5):
        B = z.size(0)
        h0 = torch.tanh(self.fc_z(z)).unsqueeze(0).repeat(2, 1, 1)
        inp = torch.full((B, 1), 0, dtype=torch.long, device=z.device)
        # start token = PAD
        outputs, h = [], h0
        for t in range(self.max_len):
            e_t = self.embedding(inp)
            out, h = self.gru(e_t, h)
            logit = self.fc_out(out)
            outputs.append(logit)
            if target_seq is not None and random.random() <
teacher_forcing_ratio:
                inp = target_seq[:, t:t+1]
            else:
                inp = logit.argmax(-1)
        return torch.cat(outputs, dim=1)

class ProteinVAE(nn.Module):
    def __init__(self, vocab_size=VOCAB_SIZE, embed_dim=64,
hidden_dim=256,
                    latent_dim=LATENT_DIM, max_len=MAX_LEN):
        super().__init__()
        self.encoder = ProteinEncoder(vocab_size, embed_dim,
hidden_dim, latent_dim)
        self.decoder = ProteinDecoder(latent_dim, hidden_dim,
vocab_size, max_len, embed_dim)
        self.latent_dim = latent_dim

    def reparameterize(self, mu, log_var):
        return mu + torch.exp(0.5 * log_var) * torch.randn_like(mu)

    def forward(self, x):
        mu, log_var = self.encoder(x)
        z = self.reparameterize(mu, log_var)
        recon = self.decoder(z, target_seq=x)
        return recon, mu, log_var

    @torch.no_grad()
    def generate(self, n=8):
        self.eval()
        z = torch.randn(n, self.latent_dim,
device=next(self.parameters()).device)
        logits = self.decoder(z)
        seqs = []
        for row in logits.argmax(-1):
            seq = ''.join([idx2aa.get(i.item(), '') for i in row

```

```

        if idx2aa.get(i.item(), '') not in
(PAD_TOKEN, UNK_TOKEN, '')]]
        seqs.append(seq)
    return seqs

vae_model = ProteinVAE().to(DEVICE)
print(f'VAE params: {sum(p.numel() for p in vae_model.parameters() if
p.requires_grad):,}')

VAE params: 2,492,182

# — CELL 15: Train VAE —————
def vae_loss(recon_logits, targets, mu, log_var, beta=1.0):
    B, L, V = recon_logits.shape
    recon = F.cross_entropy(recon_logits.reshape(B*L, V),
targets.reshape(B*L), ignore_index=0)
    kl = -0.5 * torch.mean(1 + log_var - mu.pow(2) - log_var.exp())
    return recon + beta * kl, recon.item(), kl.item()

def train_vae(vae, loader, epochs=15, lr=1e-3):
    optimizer = torch.optim.AdamW(vae.parameters(), lr=lr,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=epochs)
    scaler = torch.cuda.amp.GradScaler()
    history = {'total': [], 'recon': [], 'kl': []}

    for epoch in range(1, epochs+1):
        vae.train()
        ep_total, ep_recon, ep_kl, n = 0, 0, 0, 0
        beta_t = min(1.0, epoch / (epochs * 0.5))
        for x in tqdm(loader, leave=False):
            x = x.to(DEVICE)
            optimizer.zero_grad()
            with torch.cuda.amp.autocast():
                recon, mu, log_var = vae(x)
                loss, rl, kl = vae_loss(recon, x, mu, log_var,
beta=beta_t)
            scaler.scale(loss).backward()
            torch.nn.utils.clip_grad_norm_(vae.parameters(), 1.0)
            scaler.step(optimizer); scaler.update()
            ep_total += loss.item(); ep_recon += rl; ep_kl += kl; n +=
1

        scheduler.step()
        history['total'].append(ep_total/n)
        history['recon'].append(ep_recon/n)
        history['kl'].append(ep_kl/n)
        print(f'VAE Epoch {epoch:02d}/{epochs} | Total:
{ep_total/n:.4f} | Recon: {ep_recon/n:.4f} | KL: {ep_kl/n:.4f} | beta:
{beta_t:.3f}')
```

```

    torch.save(vae.state_dict(), 'vae_best.pt')
    return history

print('='*60)
print('Training Protein VAE...')
print('='*60)
vae_history = train_vae(vae_model, vae_loader, epochs=15)

/tmp/ipykernel_5168/776885785.py:11: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = torch.cuda.amp.GradScaler()

=====
Training Protein VAE...
=====

0%|          | 0/441 [00:00<?,
?it/s]/tmp/ipykernel_5168/776885785.py:21: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast():

VAE Epoch 01/15 | Total: 2.8518 | Recon: 2.8495 | KL: 0.0168 | beta:
0.133

VAE Epoch 02/15 | Total: 2.8066 | Recon: 2.7956 | KL: 0.0412 | beta:
0.267

VAE Epoch 03/15 | Total: 2.7635 | Recon: 2.7427 | KL: 0.0519 | beta:
0.400

VAE Epoch 04/15 | Total: 2.7006 | Recon: 2.6731 | KL: 0.0515 | beta:
0.533

VAE Epoch 05/15 | Total: 2.6207 | Recon: 2.5891 | KL: 0.0474 | beta:
0.667

VAE Epoch 06/15 | Total: 2.5723 | Recon: 2.5370 | KL: 0.0441 | beta:
0.800

```


VAE Epoch 07/15 | Total: 2.5381 | Recon: 2.5000 | KL: 0.0407 | beta: 0.933

VAE Epoch 08/15 | Total: 2.5034 | Recon: 2.4642 | KL: 0.0391 | beta: 1.000

VAE Epoch 09/15 | Total: 2.4740 | Recon: 2.4348 | KL: 0.0393 | beta: 1.000

VAE Epoch 10/15 | Total: 2.4478 | Recon: 2.4086 | KL: 0.0393 | beta: 1.000

VAE Epoch 11/15 | Total: 2.4346 | Recon: 2.3951 | KL: 0.0395 | beta: 1.000

VAE Epoch 12/15 | Total: 2.4263 | Recon: 2.3867 | KL: 0.0396 | beta: 1.000

VAE Epoch 13/15 | Total: 2.4201 | Recon: 2.3805 | KL: 0.0396 | beta: 1.000

VAE Epoch 14/15 | Total: 2.4164 | Recon: 2.3768 | KL: 0.0396 | beta: 1.000

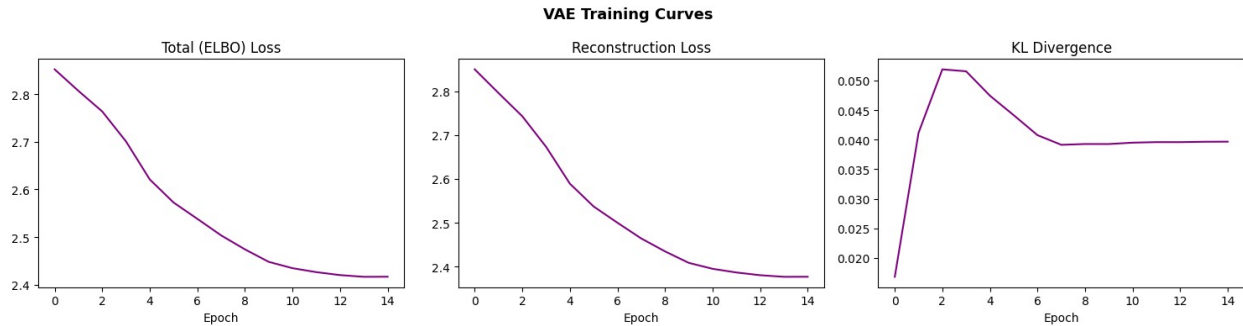
VAE Epoch 15/15 | Total: 2.4166 | Recon: 2.3770 | KL: 0.0397 | beta: 1.000

— CELL 16: VAE Results —

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, key, title in zip(axes, ['total', 'recon', 'kl'],
    ['Total (ELBO) Loss', 'Reconstruction Loss', 'KL Divergence']):
    ax.plot(vae_history[key], color='purple')
    ax.set_title(title); ax.set_xlabel('Epoch')
plt.suptitle('VAE Training Curves', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('vae_curves.png', dpi=150)
```

```
plt.show()

gen_seqs = vae_model.generate(n=10)
print('\nVAE Generated Sequences:')
for i, s in enumerate(gen_seqs):
    print(f' {i+1:2d}. {s[:80]}... (len={len(s)})')
```



VAE Generated Sequences:

1.
KIIIAAKDNGKLLSEELAEALEELLAADDVAAAAAALLLALLFFRSTRTRTSEEAALLLGGGAVLLLSS
SSSSSSGGEE... (len=256)
2.
AIVGKRLSGDELLDELLEELAEELRAAGIALAAVVVDGVSLTVVRVEEEAAAAALGKELLEVEEELEEL
LALLLELLEE... (len=256)
3.
LLLLLASGGRRLGDDLAAALADAAAAAADVGGVSLAAAALAARRLGEEVEEAAAAAALAAREGVVVVVV
GDDAAVAEE... (len=256)
4.
LELLLLLGGHSLSSSAAAAALLSSSSSSSRQILELLELLLLLFRRRGRRTLTTTGGGLLRLLGAAVL
LADSSSSGG... (len=256)
5.
LELRAFFAAVARLGFSFAAAEALLSPAAVSRRLRELEELLGLLLLRRRRRRRLTLTGEGELLRLLAAAA
AAAAAALLA... (len=256)
6.
QHLLTLDGHSLTDDLAAAAALALLASGGGDVPALAGAALAALFFETTRTRTSFEEAALLLGGGAVLLLSS
SSSSSSGGEE... (len=256)
7.
KLEELVLLVAEETPSGAVALLDEAAGILTTGGGGTSHAAVVARRLGIPAVVGAAALALADGGAVVVDGG
GGGGGGGIG... (len=256)
8.
VLELLGGHGGGLGAAAAAAAAARLLAAGVAVVVVDGGGGGALALALALAAAAAAAAAAAAAALAAAAALA
ALAALALLAL... (len=256)
9.
AALFALFRAGLLTFGGGPAALPLLREEVVVRGWLTPEFFDGLAAAAQLPGPGPAAAAAVGGLLGGGGGG
AAAAAALGLL... (len=256)
- 10.

```
ILDLGCGTGRLAALLAERGGEVTGVDLSPEMLEAARRAAAEGLVVEVDAADLDLPPDDDFDFVSSVVLH  
HLPDPEAAAA... (len=256)
```

```
# — CELL 17: VAE Latent Space t-SNE —————
```

```
vae_model.eval()  
vis_df = train_df.sample(min(3000, len(train_df)), random_state=SEED)
```

```
vis_loader = DataLoader(  
    ProteinDataset(vis_df),  
    batch_size=128, shuffle=False, num_workers=2, pin_memory=True  
)
```

```
all_mus, alllbls = [], []  
with torch.no_grad():  
    for x, y in tqdm(vis_loader, desc="Extracting latents"):  
        mu, _ = vae_model.encoder(x.to(DEVICE))  
        all_mus.append(mu.cpu().numpy())  
        alllbls.extend(y.numpy())
```

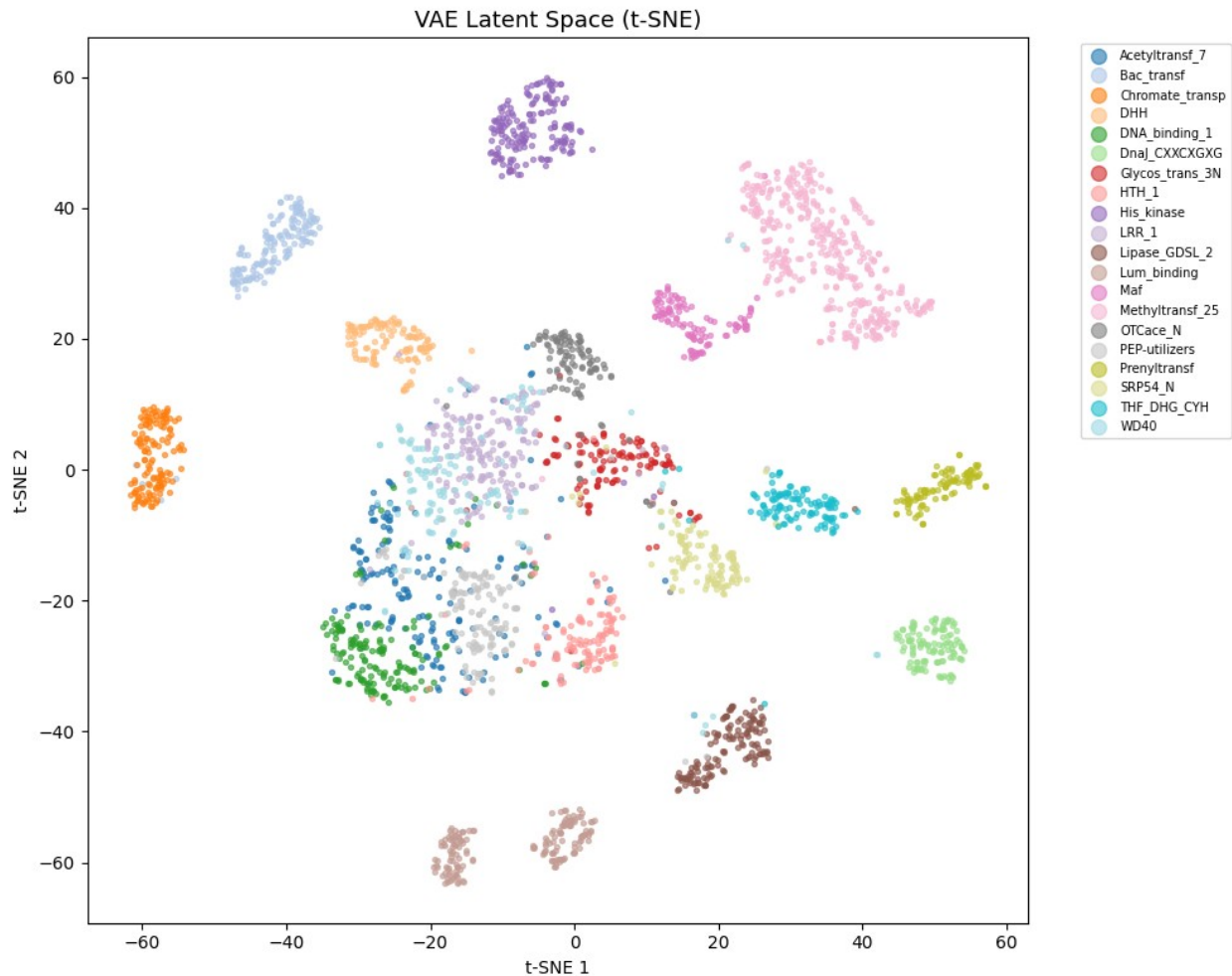
```
all_mus = np.vstack(all_mus)  
alllbls = np.array(alllbls)
```

```
print('Running t-SNE...')  
mus_2d = TSNE(n_components=2, perplexity=30, random_state=SEED,  
n_iter=1000).fit_transform(all_mus)
```

```
plt.figure(figsize=(10, 8))  
palette = sns.color_palette('tab20', NUM_CLASSES)  
for i in range(NUM_CLASSES):  
    mask = alllbls == i  
    plt.scatter(mus_2d[mask, 0], mus_2d[mask, 1],  
                c=[palette[i]], label=label_enc.classes_[i][:15],  
                alpha=0.6, s=8)
```

```
plt.legend(markerscale=3, fontsize=7, bbox_to_anchor=(1.05, 1))  
plt.title('VAE Latent Space (t-SNE)', fontsize=13)  
plt.xlabel('t-SNE 1'); plt.ylabel('t-SNE 2')  
plt.tight_layout()  
plt.savefig('vae_tsne.png', dpi=150, bbox_inches='tight')  
plt.show()
```

```
Extracting latents: 100%|██████████| 24/24 [00:01<00:00, 18.91it/s]  
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:116:  
4: FutureWarning: 'n_iter' was renamed to 'max_iter' in version 1.5  
and will be removed in 1.7.  
    warnings.warn(  
Running t-SNE...
```



```
# — CELL 18: Latent Space Interpolation —————
vae_model.eval()

# Pick two sequences from different families if possible
idx_a = 0
idx_b = next(i for i in range(100, len(train_df)) if
train_df['label'].iloc[i] != train_df['label'].iloc[0])

seq_a = train_df['sequence'].iloc[idx_a]
seq_b = train_df['sequence'].iloc[idx_b]

with torch.no_grad():
    xa = encode_sequence(seq_a).unsqueeze(0).to(DEVICE)
    xb = encode_sequence(seq_b).unsqueeze(0).to(DEVICE)
    mu_a, _ = vae_model.encoder(xa)
    mu_b, _ = vae_model.encoder(xb)

print(f'Sequence A ({train_df["family_id"].iloc[idx_a]}):
{seq_a[:60]}...')
print(f'Sequence B ({train_df["family_id"].iloc[idx_b]}):
```

```
{seq_b[:60]}...'')
print('\nInterpolation (alpha 0.0 → 1.0):')

with torch.no_grad():
    for alpha in [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]:
        z = (1 - alpha) * mu_a + alpha * mu_b
        logits = vae_model.decoder(z)
        seq_idx = logits.argmax(-1)[0]
        s = ''.join([idx2aa.get(i.item(), '') for i in seq_idx
                      if idx2aa.get(i.item(), '') not in (PAD_TOKEN,
UNK_TOKEN, '')])
        print(f'  alpha={alpha:.1f}: {s[:80]}... (len={len(s)})')
```

Sequence A (His_kinase):

SEIKLLHAQVNPFLFNALNTLSAVIRRDPEKACHLVNLSTFFRKNLKRPS EEAALSEE...

Sequence B (Maf):

KIILASNSPRRQELLRGLEVEFEVKVNPVDEKIPSDMKPEYVAAYLSKKKADAYPDELAE...

Interpolation (alpha 0.0 → 1.0):

```
alpha=0.0:
AELKLLQQQINPHFLFNTLNTISALARDDPEAAAELLDELLRLLRYLLSSSRTRLLLEEEELLNYYLL
QRRRRRGRFL... (len=256)
alpha=0.2:
AELKLLQQQINPHFLFNTLNTISALARDDPEEAELLSSLLSLLRYLLSSNNEEVLLLEEELELLRYYLE
LQRRFGRRL... (len=256)
alpha=0.4:
KEILALQQQINPHFLFNTLNTISALAENDDEEAELLSSLLSLLRYLLSSNNEEITLLEEELELLRYYLE
LQRRFGERL... (len=256)
alpha=0.6:
KIILASSSPRRRELLKQLGIPFEIIPSPDEEEIPDEPPEEEELAELLKKAACAAAAAAAEKIKLIIDDDTI
VVGDDIIKP... (len=256)
alpha=0.8:
KIILASSSPRRRELLQQLGIPFEIIPSPDEEEIPDEPPEEELAKLLAKKAAAAAAAEKPLIIIGDDDI
VVGGGIILKK... (len=256)
alpha=1.0:
KIILASSSPRRRELLKQLGIPFEIIPSPDEDEIPEEEPEEELEELLKKAACAAAAAAAEKIKIIIGDDDI
IVVGDDIIK... (len=256)
```

— CELL 19: Conditional GAN Setup —

CGAN_N_CLASSES = NUM_CLASSES

NOISE_DIM = 128

```
class ProteinCondDataset(Dataset):
    def __init__(self, df):
        self.sequences = df['sequence'].values
        self.labels = df['label'].values
    def __len__(self): return len(self.sequences)
    def __getitem__(self, idx):
        return (encode_sequence(self.sequences[idx]),
                torch.tensor(self.labels[idx], dtype=torch.long))
```

```

cgan_loader = DataLoader(
    ProteinCondDataset(train_df),
    batch_size=64, shuffle=True, num_workers=2, pin_memory=True
)

print(f'cGAN training samples: {len(train_df):,} | Classes: {CGAN_N_CLASSES}')

cGAN training samples: 28,179 | Classes: 20

# — CELL 20: cGAN Architecture —————
class CGANGenerator(nn.Module):
    def __init__(self, noise_dim, n_classes, vocab_size, max_len,
                  hidden_dim=256, embed_dim=32):
        super().__init__()
        self.max_len = max_len
        self.hidden_dim = hidden_dim
        self.label_emb = nn.Embedding(n_classes, embed_dim)
        self.fc_in = nn.Linear(noise_dim + embed_dim, hidden_dim
* max_len)
        self.net = nn.Sequential(
            nn.Conv1d(hidden_dim, 512, 3, padding=1),
            nn.BatchNorm1d(512), nn.LeakyReLU(0.2),
            nn.Conv1d(512, 256, 3, padding=1), nn.BatchNorm1d(256),
            nn.LeakyReLU(0.2),
            nn.Conv1d(256, vocab_size, 3, padding=1)
        )

    def forward(self, z, labels):
        c = self.label_emb(labels)
        h = self.fc_in(torch.cat([z, c], dim=-1))
        h = h.view(h.size(0), self.hidden_dim, self.max_len)
        return self.net(h)

    @torch.no_grad()
    def generate_sequences(self, n, label_id):
        self.eval()
        z = torch.randn(n, NOISE_DIM,
device=next(self.parameters()).device)
        labels = torch.full((n,), label_id, dtype=torch.long,
device=next(self.parameters()).device)
        logits = self.forward(z, labels)
        idx = logits.argmax(1)
        seqs = []
        for row in idx:
            seq = ''.join([idx2aa.get(i.item(), '') for i in row
                            if idx2aa.get(i.item(), '') not in
(PAD_TOKEN, UNK_TOKEN, '')])
            seqs.append(seq)

```

```

        return seqs

class CGANDiscriminator(nn.Module):
    def __init__(self, vocab_size, n_classes, max_len,
                  embed_dim=64, label_embed_dim=32):
        super().__init__()
        self.label_emb = nn.Embedding(n_classes, label_embed_dim)
        self.seq_emb = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        in_ch = embed_dim + label_embed_dim
        self.net = nn.Sequential(
            nn.Conv1d(in_ch, 256, 4, stride=2, padding=1),
nn.LeakyReLU(0.2),
            nn.Conv1d(256, 512, 4, stride=2, padding=1),
nn.LeakyReLU(0.2),
            nn.Conv1d(512, 512, 4, stride=2, padding=1),
nn.LeakyReLU(0.2),
            nn.AdaptiveAvgPool1d(1)
        )
        self.fc = nn.Linear(512, 1)

    def forward(self, x, labels):
        e_seq = self.seq_emb(x).permute(0, 2, 1)
        e_lbl = self.label_emb(labels).unsqueeze(-1).expand(-1, -1,
x.size(1))
        h = self.net(torch.cat([e_seq, e_lbl], dim=1)).squeeze(-1)
        return self.fc(h)

G = CGANGenerator(NOISE_DIM, CGAN_N_CLASSES, VOCAB_SIZE,
MAX_LEN).to(DEVICE)
D = CGANDiscriminator(VOCAB_SIZE, CGAN_N_CLASSES, MAX_LEN).to(DEVICE)

print(f'Generator params:      {sum(p.numel() for p in G.parameters()
if p.requires_grad):,}')
print(f'Discriminator params: {sum(p.numel() for p in D.parameters()
if p.requires_grad):,}')

Generator params:      11,357,590
Discriminator params:  1,675,009

# — CELL 21: Train cGAN —————
def train_cgan(G, D, loader, epochs=25):
    opt_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5,
0.999))
    opt_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5,
0.999))
    history = {'d_loss': [], 'g_loss': []}

```

```

    for epoch in range(1, epochs+1):
        ep_d, ep_g, n = 0, 0, 0
        G.train(); D.train()
        for real_x, labels in tqdm(loader, leave=False, desc=f"Epoch
{epoch}"):
            real_x, labels = real_x.to(DEVICE), labels.to(DEVICE)
            B = real_x.size(0)

            # Train Discriminator
            z = torch.randn(B, NOISE_DIM, device=DEVICE)
            fake_logits = G(z, labels)
            fake_x = fake_logits.argmax(1).detach()

            d_real = D(real_x, labels)
            d_fake = D(fake_x, labels)
            d_loss = (F.relu(1.0 - d_real) + F.relu(1.0 +
d_fake)).mean()

            opt_D.zero_grad()
            d_loss.backward()
            opt_D.step()

            # Train Generator
            z = torch.randn(B, NOISE_DIM, device=DEVICE)
            fake_x = G(z, labels).argmax(1)
            g_loss = -D(fake_x, labels).mean()

            opt_G.zero_grad()
            g_loss.backward()
            opt_G.step()

            ep_d += d_loss.item()
            ep_g += g_loss.item()
            n += 1

        history['d_loss'].append(ep_d/n)
        history['g_loss'].append(ep_g/n)

        if epoch % 5 == 0 or epoch == 1:
            print(f'cGAN Epoch {epoch:03d}/{epochs} | D: {ep_d/n:.4f}
| G: {ep_g/n:.4f}')

        torch.save(G.state_dict(), 'cgan_G.pt')
        torch.save(D.state_dict(), 'cgan_D.pt')
        return history

print('='*60)
print('Training Conditional GAN...')
print('='*60)
cgan_history = train_cgan(G, D, cgan_loader, epochs=25)

```



```
=====
Training Conditional GAN...
=====
```

```
cGAN Epoch 001/25 | D: 0.0235 | G: 2.3451
```

```
cGAN Epoch 005/25 | D: 0.0000 | G: 20.7598
```

```
cGAN Epoch 010/25 | D: 0.0000 | G: 20.7492
```

```
cGAN Epoch 015/25 | D: 0.0000 | G: 20.7519
```

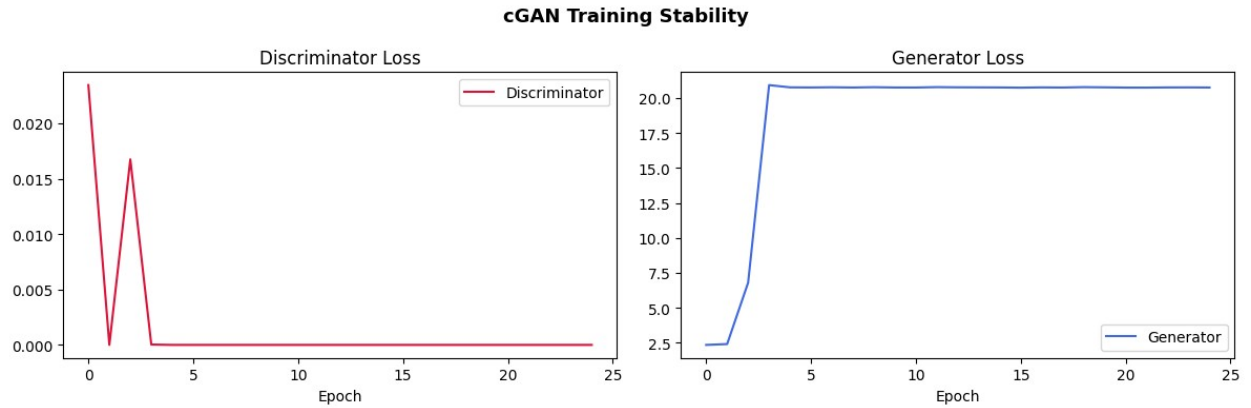
```
cGAN Epoch 020/25 | D: 0.0000 | G: 20.7607
```

```
cGAN Epoch 025/25 | D: 0.0000 | G: 20.7481
```

```
# — CELL 22: cGAN Results —
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(cgan_history['d_loss'], color='crimson',
label='Discriminator')
axes[1].plot(cgan_history['g_loss'], color='royalblue',
label='Generator')
for ax in axes:
    ax.set_xlabel('Epoch')
    ax.legend()
axes[0].set_title('Discriminator Loss')
axes[1].set_title('Generator Loss')
plt.suptitle('cGAN Training Stability', fontsize=13,
fontweight='bold')
plt.tight_layout()
plt.savefig('cgan_curves.png', dpi=150)
plt.show()
```

```
print('\ncGAN Generated Sequences (first 5 families):')
for cls in range(min(5, CGAN_N_CLASSES)):
    seqs = G.generate_sequences(n=3, label_id=cls)
    print(f'\n Family: {label_enc.classes_[cls]}')
    for s in seqs:
        print(f'      {s[:70]}... (len={len(s)})')
```



cGAN Generated Sequences (first 5 families):

Family: Acetyltransf_7

ISICQVIIRRIIWMIHPPISIIHIIIRQIQIILIEIIIMWIRPYIRRIPIRIIIIPQIIIIQISIKLI
... (len=248)

MIISWISITIISSVWIIPIQRILIMIWWIPSNISIAIIIIIRSYWQWISISRIIRDRQIIIIQIMYIMSW
... (len=239)

IISIQWQIIQYIIISWQMFIDFVIHSRRLQIQGISSQSIQIIISGQYNMSMWRRIRIQQSRSMMIIMSI
... (len=243)

Family: Bac_transf

AIVIIQSSIYPQSWIIKWSYEWIWIEMRQQIQIQIIPPGIESIPDSIQWPIHQIIWIIQIRQQIWQIIQR
... (len=245)

HYVYIRDIIISIVIIIISSYMNIIMILQIALIIRGISLWISSWIPVISQRIFMYWIPTQWIIPRIIIHI
... (len=245)

IQQWSYIDIIRMGQIDYRSESIPIIRMISIIIRQPIKIRMLIHIPGSWIIRIISIIMSISIAISLIWIYSI
... (len=243)

Family: Chromate_transp

WRIIMPIQPQDNHWIGLISSVQMMIMIIIESIILIIDRIIQDEEIYSISMYKILIIIRISQMIWYMNSIS
... (len=243)

PIIVIIMIRGIQYQIIMIIGRLWHIIIEIIILMIQLIIAISPGYLEIQMRDQQMYRIIYIIGQMIIIPMM
... (len=245)

IIIIQLIIEIRPRMLISQIWWKYSSSIYIIQIMSIPIPIQIPVEWIIIIISGGIWIISTLIASMIVII
... (len=245)

Family: DHH

```
PIYIDMKRGYVWRHRQSWISILMLGMISQLAIMIIIIIGIISRQIRIIIIIIINSPQYSSIMIIICIMIQISV
... (len=250)
```

```
MSSLVQIIIIQYQWWPIQIIIPQQIIVSMVMIMIIPPQSRIPSIIQSQSISSNEMIYMISYRQSIRQSWI
... (len=242)
```

```
INQIMQKIISIIIIHMIRISHSIQIFIMEMIISIIRLIIQDVIMFIRPMWINVYIIIIYMLSIIMQLMIIIN
... (len=252)
```

Family: DNA_binding_1

```
MIPIIIIIILEILQIRIIMSWSRIVGQYSDQDIIIIYMISDKIMSQRLLIPIYDIPQMIIISWRDMIQVGY
... (len=246)
```

```
MIWLIPQIIDYIRIIYPLIIIIYQYKIMDIIQQIMPVDSIILWIIDMWQQGMTIIPWSSIGQNSWIRQIM
... (len=248)
```

```
MILQWIMIIIIIIQIIQKIIDGIWSWQIRMIIGISIIISWIDIISQIIISIPEKSRSQIRIESISSRIAQL
... (len=245)
```

— CELL 23: Mode Collapse Detection —————

```
print('Mode Collapse Analysis (cGAN)...')
```

```
all_gen = []
```

```
for cls in range(CGAN_N_CLASSES):
    all_gen.extend(G.generate_sequences(n=10, label_id=cls))
```

```
unique_ratio = len(set(all_gen)) / len(all_gen)
```

```
print(f'Generated: {len(all_gen)} | Unique: {len(set(all_gen))} |
Ratio: {unique_ratio:.3f}')
```

```
if unique_ratio < 0.5:
    print('WARNING: Severe mode collapse detected!')
elif unique_ratio < 0.8:
    print('WARNING: Mild mode collapse detected.')
else:
    print('No severe mode collapse detected.')
```

Mode Collapse Analysis (cGAN)...

Generated: 200 | Unique: 200 | Ratio: 1.000

No severe mode collapse detected.

— CELL 24: AA Frequency Analysis —————

```
def aa_frequency(sequences):
```

```
    counts = {aa: 0 for aa in AMINO_ACIDS}
```

```
    total = 0
```

```
    for seq in sequences:
```

```
        for aa in seq:
```

```
            if aa in counts:
```

```
                counts[aa] += 1
```

```
                total += 1
```

```
    return {aa: (c / total if total else 0) for aa, c in
```

```

counts.items()

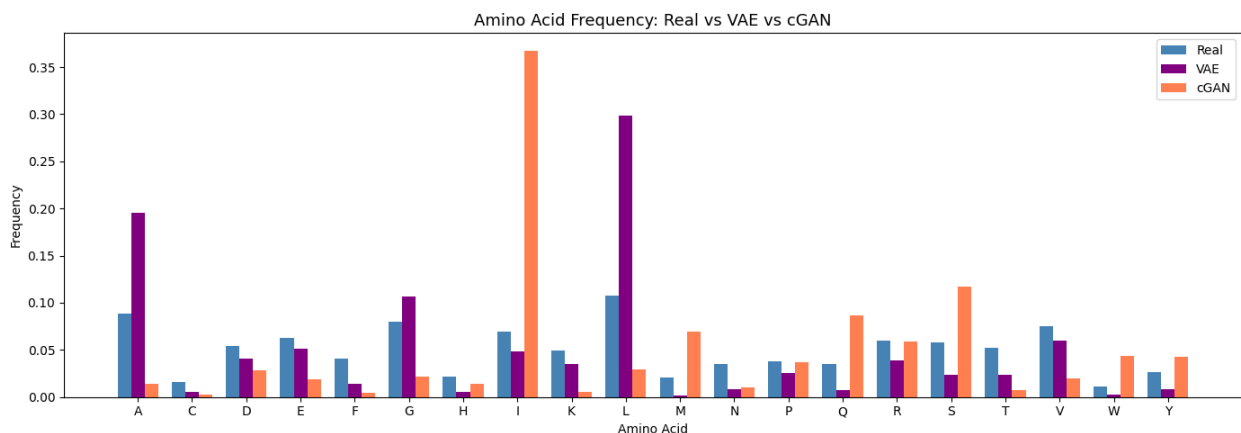
real_seqs = train_df['sequence'].sample(500,
random_state=SEED).tolist()
vae_seqs  = vae_model.generate(n=200)
cgan_seqs = G.generate_sequences(n=200, label_id=0)  # example from
class 0

real_freq = aa_frequency(real_seqs)
vae_freq  = aa_frequency(vae_seqs)
cgan_freq = aa_frequency(cgan_seqs)

x = np.arange(len(AMINO_ACIDS))
w = 0.25
fig, ax = plt.subplots(figsize=(14, 5))
ax.bar(x - w, [real_freq[a] for a in AMINO_ACIDS], w, label='Real',
color='steelblue')
ax.bar(x,      [vae_freq[a]  for a in AMINO_ACIDS], w, label='VAE',
color='purple')
ax.bar(x + w, [cgan_freq[a] for a in AMINO_ACIDS], w, label='cGAN',
color='coral')

ax.set_xticks(x)
ax.set_xticklabels(AMINO_ACIDS)
ax.set_title('Amino Acid Frequency: Real vs VAE vs cGAN', fontsize=13)
ax.set_xlabel('Amino Acid')
ax.set_ylabel('Frequency')
ax.legend()
plt.tight_layout()
plt.savefig('aa_frequency.png', dpi=150)
plt.show()

```



```

# — CELL 25: Final Summary —————
print('='*70)
print('FINAL RESULTS SUMMARY - DeepGen-Protein')

```

```

print('='*70)

print('\nISA-1: Protein Family Classification')
print(f' {"Model":<10} {"Accuracy":>10} {"Macro-F1":>10}')
print(' ' + '-'*32)
for name in results:
    print(f' {name:<10} {results[name]["accuracy"]:>10.4f}
{results[name]["f1_macro"]:>10.4f}')

print('\nISA-2: Generative Models')
print(f' {"Model":<10} {"Samples":>8} {"Unique":>8} {"AA
Entropy":>12}')
print(' ' + '-'*42)
for name, seqs in [('VAE', vae_seqs), ('cGAN', cgan_seqs)]:
    freq = aa_frequency(seqs)
    entropy = -sum(p * np.log(p + 1e-9) for p in freq.values())
    print(f' {name:<10} {len(seqs):>8} {len(set(seqs)):>8}
{entropy:>12.3f}')

print('\nAll models trained and evaluated successfully!')
print('Saved models:')
print(' • CNN1D_best.pt, LSTM_best.pt, GRU_best.pt')
print(' • vae_best.pt')
print(' • cgan_G.pt, cgan_D.pt')
print('Plots saved in working directory.')

```

FINAL RESULTS SUMMARY - DeepGen-Protein

ISA-1: Protein Family Classification

Model	Accuracy	Macro-F1
1D-CNN	0.9994	0.9994
LSTM	0.9974	0.9972
GRU	0.9994	0.9993

ISA-2: Generative Models

Model	Samples	Unique	AA Entropy
VAE	200	200	2.287
cGAN	200	200	2.289

All models trained and evaluated successfully!

Saved models:

- CNN1D_best.pt, LSTM_best.pt, GRU_best.pt
- vae_best.pt
- cgan_G.pt, cgan_D.pt

Plots saved in working directory.

** DeepGen-Protein: Generative Models on UniProt/Swiss-Prot**

```
# — CELL 1: Imports & Setup —————
import os, random, math, re, gzip
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from sklearn.manifold import TSNE

SEED = 42
random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
if DEVICE.type == 'cuda':
    print(f'GPU: {torch.cuda.get_device_name(0)}')

Device: cuda
GPU: Tesla T4

# — CELL 2: Load UniProt Swiss-Prot FASTA —————
# Supports both .fasta and .fasta.gz
FASTA_PATH = '/content/uniprot_sprot.fasta.gz' # change if needed

def parse_fasta(path, max_seqs=50000):
    """Parse FASTA file, return list of (accession, organism,
    sequence) tuples."""
    records = []
    open_fn = gzip.open if path.endswith('.gz') else open
    mode = 'rt'
    seq, header = '', ''
    with open_fn(path, mode) as f:
        for line in f:
            line = line.strip()
            if line.startswith('>'):
                if seq and header:
                    records.append(_parse_header(header, seq))
                    if len(records) >= max_seqs:
                        break
                header, seq = line, ''
```

```

        else:
            seq += line
        if seq and header and len(records) < max_seqs:
            records.append(_parse_header(header, seq))
    return records

def _parse_header(header, seq):
    # Swiss-Prot format: >sp|ACC|ENTRY_NAME ... OS=Organism OX=...
    GN=... PE=... SV=...
    parts = header.split('|')
    acc = parts[1] if len(parts) > 1 else 'UNK'
    os_match = re.search(r'OS=(^[^=]+?)(?:\s+OX=|\s+GN=|\s+PE=|$',
header)
    organism = os_match.group(1).strip() if os_match else 'Unknown'
    return {'accession': acc, 'organism': organism, 'sequence': seq}

print('Parsing FASTA... (this may take ~30s for full Swiss-Prot)')
records = parse_fasta(FASTA_PATH, max_seqs=50000)
df = pd.DataFrame(records)
print(f'Loaded {len(df):,} sequences')
print(f'Columns: {df.columns.tolist()}')
df.head()

```

Parsing FASTA... (this may take ~30s for full Swiss-Prot)

Loaded 50,000 sequences

Columns: ['accession', 'organism', 'sequence']

```

{"summary":{"name": "df", "rows": 50000, "fields":
[\n      \n      "column": "accession", \n      "properties": {\n
"dtype": "string", \n      "num_unique_values": 50000, \n
"samples": [\n      "Q1DP17", \n      "P83969", \n
"P26708", \n      ], \n      "semantic_type": "\"", \n
"description": "\"\" \n      }, \n      { \n      "column":
"organism", \n      "properties": {\n      "dtype":
"category", \n      "num_unique_values": 3809, \n
"samples": [\n      "Halalkalibacterium halodurans (strain ATCC
BAA-125 / DSM 18197 / FERM 7344 / JCM 9153 / C-125)", \n
"Simian foamy virus (isolate chimpanzee)", \n      "Rana
shuchinae", \n      ], \n      "semantic_type": "\"", \n
"description": "\"\" \n      }, \n      { \n      "column":
"sequence", \n      "properties": {\n      "dtype":
"string", \n      "num_unique_values": 43355, \n
"samples": [\n
"MSWQEKIDAALDARRVADALRRRYPVAQGAGRWLVADDCQYLNFSNDYLGLSHHPQIIRAWQQGAEQ
FGVGSGSGHVSQSVAHQALEEEELAEWLGYSRALLFISGFAANQAVIAAMMAKEDRIVADRLSHASLLE
AASLSPSPLRRFAHNDVTHLARLLASPCPGQQLVVTGEGVFSMDGDSAPLEEQVTTQHDGWLMMVDDAHG
TGVIGEQRGSCWLQKVKPELLVVTGKGFVSGAAVLCSDTVADYLLQFARHLIYSTSMPPAQALRA
SLGVIRSDEGDTRREKLAALITRFRAGVQDLPFTLADLCSAIQPLIVGDNSTRALQLAEKLRQQGCWVTAI
RPPTVPAGTARLRLTLTAAHEMQDIDRLLEVLHGNG", \n
"MTTKDPESEHPSVTLFRQYLRICTVQPNPDYGGAITFLEERARQLGLSCQKIEVVPGFVITVLTWPGT

```

```

NPSLPSILLNSHTDVVPVFKEHWHHDPFEAFKDSEGYIYARGSQDMKSVSIQYLEAVRRLKSEGHRFPRT
IHMTFVPDEEVGGHKGMELFVKRPEFQALRAGFALDEGLANPTDAFTVFYSPWVVRVTSTGKPGHAS
RFIEDTAAEKLHKVISSILAFREKERQRLQANPHLKEGAVTSVNLTKLEGGVAYNVVPATMSASFDRVA
PDVDMKAFEKQLQRWCQEAGEGVTFEFAQKFTEPRMPTDDSDPWAAAFSGACKAMNLTLEPEIFPAATD
SRYIRAVGIPALGFSPMNRTPVLLHDHNERLHEDIFLRGVDIYTGLLSALASVPTLPGES\", \n
\"MLSVPKSYTNGKMDQVVFQKQRNRLNQCFKFNQTKYIQFRLSEEQYNKLKISGETYGLSPNLYAKK
LAQKSHLKKPYLEHDQAKSLLELSKQGSQFKSASFRKLLDEHQLLASYSKPGYPYDNAVTEVFFKYLKQ
REINRRTYHSIQEVQLSCFEYIEQFYNNYNPHSANNGLTPNQKEENYFKKNIAHLTETDIVDRLLKGSPA
LRVGYQLYQDFLYAVKERDYVSFEELLTNMIPLPEGYQTILRTFQKFLPQIKNALQQSYSNGPLECLNNH
IKVLKRNAYGFRSFYNFKLRIMIRHGNALIFN\" \n
\", \n
\"semantic_type\": \"\", \n
\"description\": \"\" \n
} \n
} \n
] \n} \n\", \"type\": \"dataframe\", \"variable_name\": \"df\"}

```

```

# — CELL 3: EDA & Filter —————
# Keep only standard amino acids, filter by length
AMINO_ACIDS = set('ACDEFGHIKLMNPQRSTVWY')
MIN_LEN, MAX_LEN_FILTER = 50, 512

def is_valid(seq):
    return MIN_LEN <= len(seq) <= MAX_LEN_FILTER and all(c in
AMINO_ACIDS for c in seq.upper())

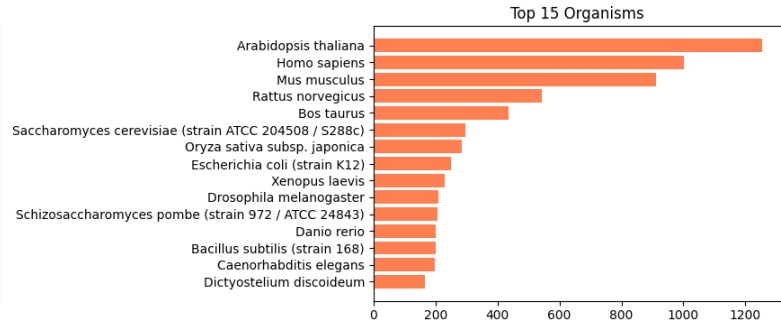
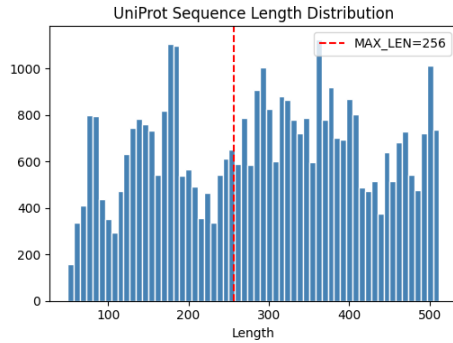
df = df[df['sequence'].apply(is_valid)].reset_index(drop=True)
print(f'After filtering: {len(df):,} sequences')

seq_lens = df['sequence'].apply(len)
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].hist(seq_lens, bins=60, color='steelblue', edgecolor='white')
axes[0].axvline(256, color='red', linestyle='--', label='MAX_LEN=256')
axes[0].set_title('UniProt Sequence Length Distribution')
axes[0].set_xlabel('Length'); axes[0].legend()

top_orgs = df['organism'].value_counts().head(15)
axes[1].barh(top_orgs.index[::-1], top_orgs.values[::-1],
color='coral')
axes[1].set_title('Top 15 Organisms')
plt.tight_layout(); plt.savefig('uniprot_eda.png', dpi=150);
plt.show()
print(f'Mean length: {seq_lens.mean():.1f} | Median:
{seq_lens.median():.1f}')

After filtering: 39,576 sequences

```

Mean length: 291.1 | Median: 297.0

```
# — CELL 4: Amino Acid Encoding —————
AA_LIST = list('ACDEFGHIKLMNPQRSTVWY')
PAD_TOKEN = '<PAD>'
UNK_TOKEN = '<UNK>'
vocab = [PAD_TOKEN] + AA_LIST + [UNK_TOKEN]
aa2idx = {aa: i for i, aa in enumerate(vocab)}
idx2aa = {i: aa for aa, i in aa2idx.items()}
VOCAB_SIZE = len(vocab)
MAX_LEN = 256

def encode_sequence(seq, max_len=MAX_LEN):
    seq = seq.upper()[:max_len]
    encoded = [aa2idx.get(aa, aa2idx[UNK_TOKEN]) for aa in seq]
    padded = encoded + [0] * (max_len - len(encoded))
    return torch.tensor(padded, dtype=torch.long)

class UniProtDataset(Dataset):
    def __init__(self, sequences):
        self.sequences = sequences
    def __len__(self): return len(self.sequences)
    def __getitem__(self, idx): return
        encode_sequence(self.sequences[idx])

vae_loader = DataLoader(
    UniProtDataset(df['sequence'].values),
    batch_size=128, shuffle=True, num_workers=2, pin_memory=True)
print(f'VOCAB_SIZE={VOCAB_SIZE} | MAX_LEN={MAX_LEN} | Batches:
{len(vae_loader)}')
```

VOCAB_SIZE=22 | MAX_LEN=256 | Batches: 310

```
# — CELL 5: VAE Architecture —————
LATENT_DIM = 128

class ProteinEncoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, latent_dim):
        super().__init__()
```

```

        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers=2,
                           batch_first=True, bidirectional=True,
dropout=0.2)
        self.fc_mu = nn.Linear(hidden_dim * 2, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim * 2, latent_dim)
    def forward(self, x):
        e = self.embedding(x)
        _, h = self.gru(e)
        h = torch.cat([h[-2], h[-1]], dim=-1)
        return self.fc_mu(h), self.fc_logvar(h)

class ProteinDecoder(nn.Module):
    def __init__(self, latent_dim, hidden_dim, vocab_size, max_len,
embed_dim):
        super().__init__()
        self.max_len = max_len
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.fc_z = nn.Linear(latent_dim, hidden_dim)
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers=2,
                           batch_first=True, dropout=0.2)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)
    def forward(self, z, target_seq=None, teacher_forcing_ratio=0.5):
        B = z.size(0)
        h0 = torch.tanh(self.fc_z(z)).unsqueeze(0).repeat(2, 1, 1)
        inp = torch.full((B, 1), 0, dtype=torch.long, device=z.device)
        outputs, h = [], h0
        for t in range(self.max_len):
            e_t = self.embedding(inp)
            out, h = self.gru(e_t, h)
            logit = self.fc_out(out)
            outputs.append(logit)
            if target_seq is not None and random.random() <
teacher_forcing_ratio:
                inp = target_seq[:, t:t+1]
            else:
                inp = logit.argmax(-1)
        return torch.cat(outputs, dim=1)

class ProteinVAE(nn.Module):
    def __init__(self, vocab_size=VOCAB_SIZE, embed_dim=64,
hidden_dim=256,
                    latent_dim=LATENT_DIM, max_len=MAX_LEN):
        super().__init__()
        self.encoder = ProteinEncoder(vocab_size, embed_dim,
hidden_dim, latent_dim)
        self.decoder = ProteinDecoder(latent_dim, hidden_dim,

```

```

vocab_size, max_len, embed_dim)
    self.latent_dim = latent_dim
    def reparameterize(self, mu, log_var):
        return mu + torch.exp(0.5 * log_var) * torch.randn_like(mu)
    def forward(self, x):
        mu, log_var = self.encoder(x)
        z = self.reparameterize(mu, log_var)
        recon = self.decoder(z, target_seq=x)
        return recon, mu, log_var
    @torch.no_grad()
    def generate(self, n=8, temperature=1.0):
        self.eval()
        z = torch.randn(n, self.latent_dim,
device=next(self.parameters()).device)
        if temperature != 1.0:
            z = z * temperature
        logits = self.decoder(z)
        seqs = []
        for row in logits.argmax(-1):
            seq = ''.join([idx2aa.get(i.item(), '') for i in row
                            if idx2aa.get(i.item(), '') not in
(PAD_TOKEN, UNK_TOKEN, '')])
            seqs.append(seq)
        return seqs

vae_model = ProteinVAE().to(DEVICE)
print(f'VAE params: {sum(p.numel() for p in vae_model.parameters() if
p.requires_grad):,}')

VAE params: 2,492,182

# — CELL 6: Train VAE —————
def vae_loss(recon_logits, targets, mu, log_var, beta=1.0):
    B, L, V = recon_logits.shape
    recon = F.cross_entropy(recon_logits.reshape(B*L, V),
                            targets.reshape(B*L), ignore_index=0)
    kl = -0.5 * torch.mean(1 + log_var - mu.pow(2) - log_var.exp())
    return recon + beta * kl, recon.item(), kl.item()

def train_vae(vae, loader, epochs=20, lr=1e-3):
    optimizer = torch.optim.AdamW(vae.parameters(), lr=lr,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=epochs)
    scaler = torch.amp.GradScaler('cuda')
    history = {'total': [], 'recon': [], 'kl': []}
    for epoch in range(1, epochs+1):
        vae.train()
        ep_total, ep_recon, ep_kl, n = 0, 0, 0, 0
        beta_t = min(1.0, epoch / (epochs * 0.5))

```

```

        for x in tqdm(loader, leave=False):
            x = x.to(DEVICE)
            optimizer.zero_grad()
            with torch.amp.autocast('cuda'):
                recon, mu, log_var = vae(x)
                loss, rl, kl = vae_loss(recon, x, mu, log_var,
beta=beta_t)
            scaler.scale(loss).backward()
            torch.nn.utils.clip_grad_norm_(vae.parameters(), 1.0)
            scaler.step(optimizer); scaler.update()
            ep_total += loss.item(); ep_recon += rl; ep_kl += kl; n +=
1
            scheduler.step()
            history['total'].append(ep_total/n)
            history['recon'].append(ep_recon/n)
            history['kl'].append(ep_kl/n)
            print(f'VAE Epoch {epoch:02d}/{epochs} | Total:
{ep_total/n:.4f} | '
                f'Recon: {ep_recon/n:.4f} | KL: {ep_kl/n:.4f} | beta:
{beta_t:.3f}')
            torch.save(vae.state_dict(), 'vae_uniprot.pt')
        return history

print('='*60)
print('Training Protein VAE on UniProt...')
print('='*60)
vae_history = train_vae(vae_model, vae_loader, epochs=20)

```

```

=====
Training Protein VAE on UniProt...
=====

```

```

VAE Epoch 01/20 | Total: 2.8690 | Recon: 2.8677 | KL: 0.0134 | beta:
0.100

```

```

VAE Epoch 02/20 | Total: 2.8497 | Recon: 2.8482 | KL: 0.0071 | beta:
0.200

```

```

VAE Epoch 03/20 | Total: 2.8427 | Recon: 2.8421 | KL: 0.0019 | beta:
0.300

```

```

VAE Epoch 04/20 | Total: 2.8385 | Recon: 2.8380 | KL: 0.0011 | beta:
0.400

```

VAE Epoch 05/20 | Total: 2.8351 | Recon: 2.8347 | KL: 0.0008 | beta: 0.500

VAE Epoch 06/20 | Total: 2.8326 | Recon: 2.8323 | KL: 0.0005 | beta: 0.600

VAE Epoch 07/20 | Total: 2.8297 | Recon: 2.8295 | KL: 0.0004 | beta: 0.700

VAE Epoch 08/20 | Total: 2.8276 | Recon: 2.8273 | KL: 0.0003 | beta: 0.800

VAE Epoch 09/20 | Total: 2.8256 | Recon: 2.8254 | KL: 0.0002 | beta: 0.900

VAE Epoch 10/20 | Total: 2.8250 | Recon: 2.8249 | KL: 0.0001 | beta: 1.000

VAE Epoch 11/20 | Total: 2.8231 | Recon: 2.8230 | KL: 0.0001 | beta: 1.000

VAE Epoch 12/20 | Total: 2.8218 | Recon: 2.8217 | KL: 0.0001 | beta: 1.000

VAE Epoch 13/20 | Total: 2.8206 | Recon: 2.8205 | KL: 0.0001 | beta: 1.000

VAE Epoch 14/20 | Total: 2.8188 | Recon: 2.8187 | KL: 0.0001 | beta: 1.000

VAE Epoch 15/20 | Total: 2.8186 | Recon: 2.8185 | KL: 0.0001 | beta: 1.000

VAE Epoch 16/20 | Total: 2.8175 | Recon: 2.8174 | KL: 0.0001 | beta: 1.000

VAE Epoch 17/20 | Total: 2.8171 | Recon: 2.8170 | KL: 0.0001 | beta: 1.000

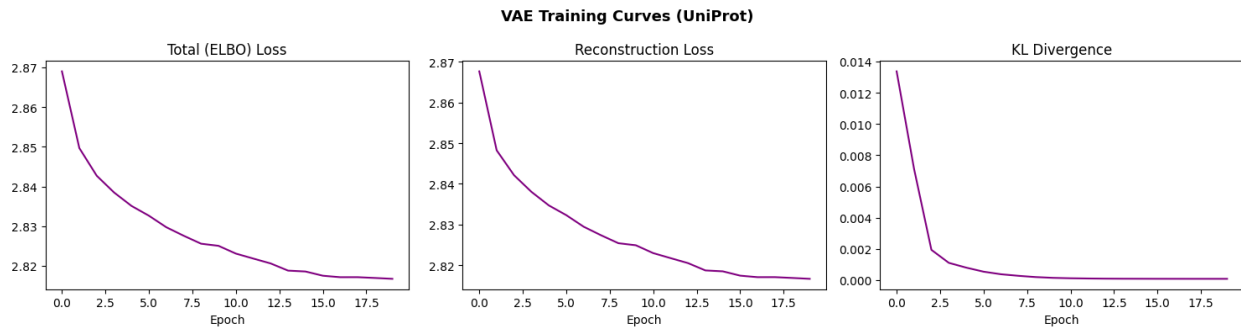
VAE Epoch 18/20 | Total: 2.8171 | Recon: 2.8170 | KL: 0.0001 | beta: 1.000

VAE Epoch 19/20 | Total: 2.8169 | Recon: 2.8168 | KL: 0.0001 | beta: 1.000

VAE Epoch 20/20 | Total: 2.8167 | Recon: 2.8166 | KL: 0.0001 | beta: 1.000

```
# — CELL 7: VAE Training Curves —————
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, key, title in zip(axes, ['total', 'recon', 'kl'],
                           ['Total (ELBO) Loss', 'Reconstruction Loss',
                            'KL Divergence']):
    ax.plot(vae_history[key], color='purple')
    ax.set_title(title); ax.set_xlabel('Epoch')
plt.suptitle('VAE Training Curves (UniProt)', fontsize=13,
             fontweight='bold')
plt.tight_layout()
plt.savefig('vae_uniprot_curves.png', dpi=150)
plt.show()

print('\nVAE Generated Sequences (UniProt):')
gen_seqs = vae_model.generate(n=10)
for i, s in enumerate(gen_seqs):
    print(f' {i+1:2d}. {s[:80]}... (len={len(s)})')
```



VAE Generated Sequences (UniProt):

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

— CELL 8: Latent Space Interpolation

```
vae_model.eval()
idx_a, idx_b = 0, 1000
seq_a = df['sequence'].iloc[idx_a]
seq_b = df['sequence'].iloc[idx_b]
```



```

vis_loader = DataLoader(UniProtDataset(vis_df['sequence'].values),
                        batch_size=128, shuffle=False, num_workers=2)
all_mus = []
vae_model.eval()
with torch.no_grad():
    for x in tqdm(vis_loader, desc='Extracting latents'):
        mu, _ = vae_model.encoder(x.to(DEVICE))
        all_mus.append(mu.cpu().numpy())
all_mus = np.vstack(all_mus)
org_labels = vis_df['organism'].values

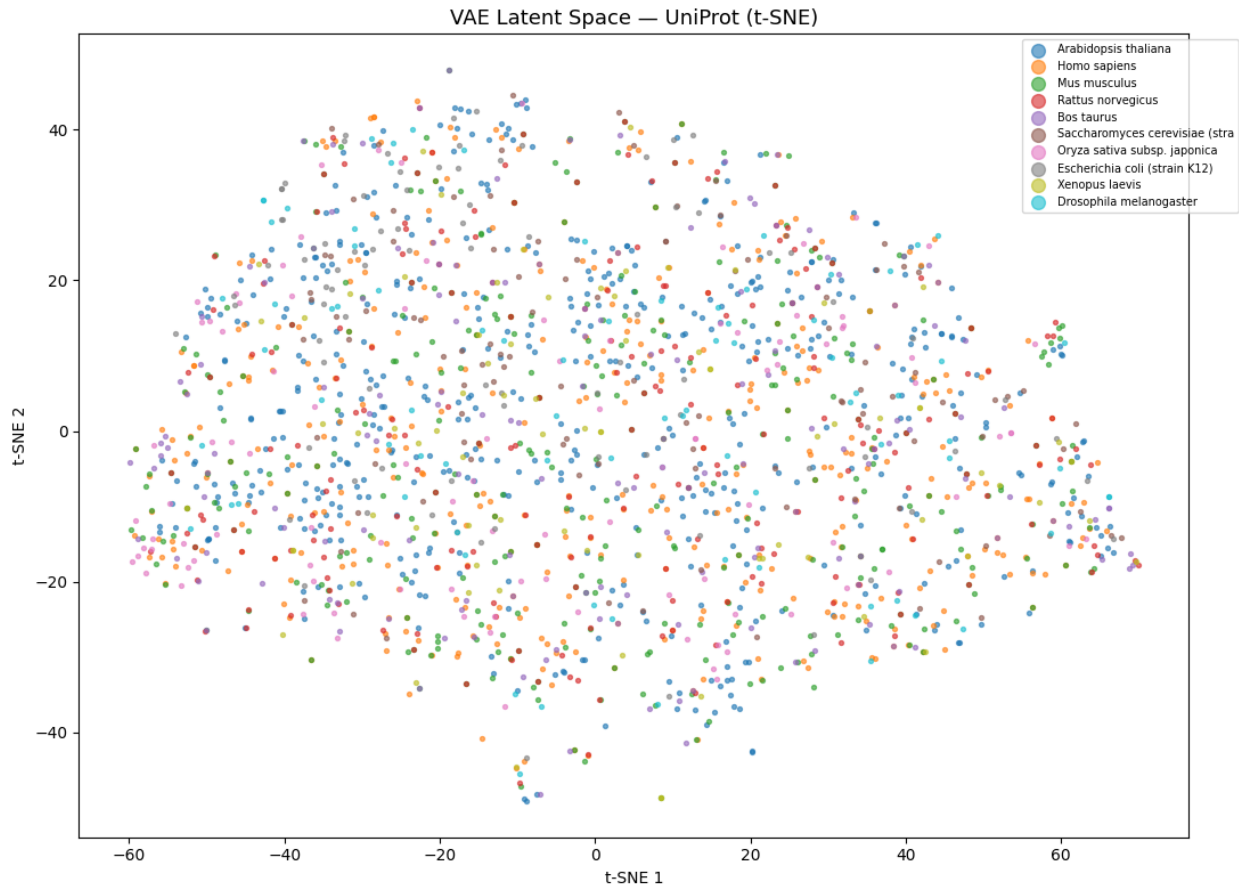
print('Running t-SNE...')
mus_2d = TSNE(n_components=2, perplexity=30, random_state=SEED,
              max_iter=1000).fit_transform(all_mus)

plt.figure(figsize=(11, 8))
palette = sns.color_palette('tab10', len(top10_orgs))
for i, org in enumerate(top10_orgs):
    mask = org_labels == org
    plt.scatter(mus_2d[mask, 0], mus_2d[mask, 1],
                c=[palette[i]], label=org[:30], alpha=0.6, s=8)
plt.legend(markerscale=3, fontsize=7, bbox_to_anchor=(1.05, 1))
plt.title('VAE Latent Space – UniProt (t-SNE)', fontsize=13)
plt.xlabel('t-SNE 1'); plt.ylabel('t-SNE 2')
plt.tight_layout()
plt.savefig('vae_uniprot_tsne.png', dpi=150, bbox_inches='tight')
plt.show()

```

Extracting latents: 100%|██████████| 16/16 [00:01<00:00, 15.48it/s]

Running t-SNE...



```
# — CELL 11: cGAN Dataset (conditioned on organism) —————
from sklearn.preprocessing import LabelEncoder

TOP_N = 10
top_orgs_list =
df['organism'].value_counts().head(TOP_N).index.tolist()
cgan_df =
df[df['organism'].isin(top_orgs_list)].reset_index(drop=True)

label_enc = LabelEncoder()
cgan_df['label'] = label_enc.fit_transform(cgan_df['organism'])
CGAN_N_CLASSES = len(label_enc.classes_)
NOISE_DIM = 128

class ProteinCondDataset(Dataset):
    def __init__(self, df):
        self.sequences = df['sequence'].values
        self.labels = df['label'].values
    def __len__(self): return len(self.sequences)
    def __getitem__(self, idx):
        return (encode_sequence(self.sequences[idx]),
                torch.tensor(self.labels[idx], dtype=torch.long))
```

```

cgan_loader = DataLoader(ProteinCondDataset(cgan_df),
                          batch_size=64, shuffle=True, num_workers=2,
                          pin_memory=True)
print(f'cGAN samples: {len(cgan_df):,} | Classes: {CGAN_N_CLASSES}')
for i, cls in enumerate(label_enc.classes_):
    print(f' [{i}] {cls}')

```

```

cGAN samples: 5,418 | Classes: 10
[0] Arabidopsis thaliana
[1] Bos taurus
[2] Drosophila melanogaster
[3] Escherichia coli (strain K12)
[4] Homo sapiens
[5] Mus musculus
[6] Oryza sativa subsp. japonica
[7] Rattus norvegicus
[8] Saccharomyces cerevisiae (strain ATCC 204508 / S288c)
[9] Xenopus laevis

```

— CELL 12: cGAN Architecture —

```

class CGANGenerator(nn.Module):
    def __init__(self, noise_dim, n_classes, vocab_size, max_len,
                  hidden_dim=256, embed_dim=32):
        super().__init__()
        self.max_len = max_len
        self.hidden_dim = hidden_dim
        self.label_emb = nn.Embedding(n_classes, embed_dim)
        self.fc_in = nn.Linear(noise_dim + embed_dim, hidden_dim
                                * max_len)
        self.net = nn.Sequential(
            nn.Conv1d(hidden_dim, 512, 3, padding=1),
            nn.BatchNorm1d(512), nn.LeakyReLU(0.2),
            nn.Conv1d(512, 256, 3, padding=1),
            nn.BatchNorm1d(256), nn.LeakyReLU(0.2),
            nn.Conv1d(256, vocab_size, 3, padding=1)
        )
    def forward(self, z, labels):
        c = self.label_emb(labels)
        h = self.fc_in(torch.cat([z, c], dim=-1))
        h = h.view(h.size(0), self.hidden_dim, self.max_len)
        return self.net(h)
    @torch.no_grad()
    def generate_sequences(self, n, label_id):
        self.eval()
        z = torch.randn(n, NOISE_DIM,
                        device=next(self.parameters()).device)
        labels = torch.full((n,), label_id, dtype=torch.long,
                           device=next(self.parameters()).device)
        idx = self.forward(z, labels).argmax(1)
        seqs = []

```

```

        for row in idx:
            seq = ''.join([idx2aa.get(i.item(), '') for i in row
                           if idx2aa.get(i.item(), '') not in
(PAD_TOKEN, UNK_TOKEN, '')])
            seqs.append(seq)
        return seqs

class CGANDiscriminator(nn.Module):
    def __init__(self, vocab_size, n_classes, max_len,
                  embed_dim=64, label_embed_dim=32):
        super().__init__()
        self.label_emb = nn.Embedding(n_classes, label_embed_dim)
        self.seq_emb = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        in_ch = embed_dim + label_embed_dim
        self.net = nn.Sequential(
            nn.Conv1d(in_ch, 256, 4, stride=2, padding=1),
nn.LeakyReLU(0.2),
            nn.Conv1d(256, 512, 4, stride=2, padding=1),
nn.LeakyReLU(0.2),
            nn.Conv1d(512, 512, 4, stride=2, padding=1),
nn.LeakyReLU(0.2),
            nn.AdaptiveAvgPool1d(1)
        )
        self.fc = nn.Linear(512, 1)
    def forward(self, x, labels):
        e_seq = self.seq_emb(x).permute(0, 2, 1)
        e_lbl = self.label_emb(labels).unsqueeze(-1).expand(-1, -1,
x.size(1))
        h = self.net(torch.cat([e_seq, e_lbl], dim=1)).squeeze(-1)
        return self.fc(h)

G = CGANGenerator(NOISE_DIM, CGAN_N_CLASSES, VOCAB_SIZE,
MAX_LEN).to(DEVICE)
D = CGANDiscriminator(VOCAB_SIZE, CGAN_N_CLASSES, MAX_LEN).to(DEVICE)
print(f'Generator:      {sum(p.numel() for p in G.parameters() if
p.requires_grad):,} params')
print(f'Discriminator: {sum(p.numel() for p in D.parameters() if
p.requires_grad):,} params')

Generator:      11,357,270 params
Discriminator: 1,674,689 params

# — CELL 13: Train cGAN —————
def train_cgan(G, D, loader, epochs=25):
    opt_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5,
0.999))
    opt_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5,
0.999))
    history = {'d_loss': [], 'g_loss': []}

```

```

for epoch in range(1, epochs+1):
    ep_d, ep_g, n = 0, 0, 0
    G.train(); D.train()
    for real_x, labels in tqdm(loader, leave=False, desc=f'Epoch
{epoch}'):
        real_x, labels = real_x.to(DEVICE), labels.to(DEVICE)
        B = real_x.size(0)
        # Train D
        z = torch.randn(B, NOISE_DIM, device=DEVICE)
        fake_x = G(z, labels).argmax(1).detach()
        d_real = D(real_x, labels)
        d_fake = D(fake_x, labels)
        d_loss = (F.relu(1.0 - d_real) + F.relu(1.0 +
d_fake)).mean()
        opt_D.zero_grad(); d_loss.backward(); opt_D.step()
        # Train G
        z = torch.randn(B, NOISE_DIM, device=DEVICE)
        fake_x = G(z, labels).argmax(1)
        g_loss = -D(fake_x, labels).mean()
        opt_G.zero_grad(); g_loss.backward(); opt_G.step()
        ep_d += d_loss.item(); ep_g += g_loss.item(); n += 1
        history['d_loss'].append(ep_d/n)
        history['g_loss'].append(ep_g/n)
        if epoch % 5 == 0 or epoch == 1:
            print(f'cGAN Epoch {epoch:03d}/{epochs} | D: {ep_d/n:.4f}
| G: {ep_g/n:.4f}')
            torch.save(G.state_dict(), 'cgan_G_uniprot.pt')
            torch.save(D.state_dict(), 'cgan_D_uniprot.pt')
    return history

print('='*60)
print('Training Conditional GAN on UniProt...')
print('='*60)
cgan_history = train_cgan(G, D, cgan_loader, epochs=15)

```

```

=====
Training Conditional GAN on UniProt...
=====

```

```
cGAN Epoch 001/25 | D: 0.1175 | G: 1.7658
```

```
cGAN Epoch 005/25 | D: 0.0000 | G: 3.0242
```

```
cGAN Epoch 010/25 | D: 0.0012 | G: 3.4006
```

cGAN Epoch 015/25 | D: 0.0000 | G: 6.9981

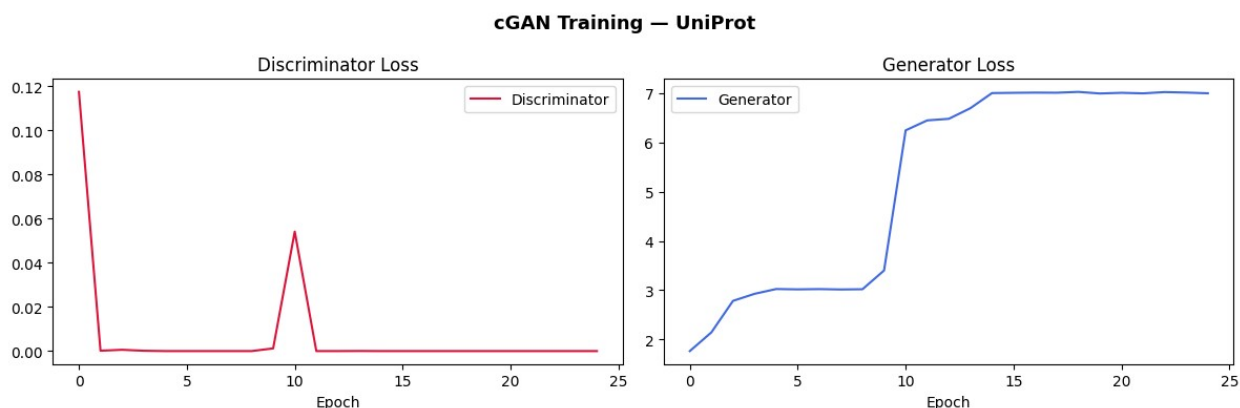
cGAN Epoch 020/25 | D: 0.0000 | G: 6.9897

cGAN Epoch 025/25 | D: 0.0000 | G: 6.9939

— *CELL 14: cGAN Results & Generation* —

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(cgan_history['d_loss'], color='crimson',
label='Discriminator')
axes[1].plot(cgan_history['g_loss'], color='royalblue',
label='Generator')
for ax in axes:
    ax.set_xlabel('Epoch'); ax.legend()
axes[0].set_title('Discriminator Loss')
axes[1].set_title('Generator Loss')
plt.suptitle('cGAN Training — UniProt', fontsize=13,
fontweight='bold')
plt.tight_layout()
plt.savefig('cgan_uniprot_curves.png', dpi=150)
plt.show()
```

```
print('\ncGAN Generated Sequences (per organism):')
for cls_id in range(CGAN_N_CLASSES):
    seqs = G.generate_sequences(n=2, label_id=cls_id)
    print(f'\n Organism: {label_enc.classes_[cls_id]}')
    for s in seqs:
        print(f'    {s[:70]}... (len={len(s)})')
```



cGAN Generated Sequences (per organism):

Organism: Arabidopsis thaliana

PGFWFIEWRTYHHFPECIGHGWFGYHYWWYEFMHGMGTLLHINFFAIGYYWPEYFFISTILLMYICLHFY
... (len=237)

GHIWVGHGLMHHLLYGYFRTYICHIHSLIEIMAHWMWLYMIYWWNYMYWYHYKMIGTCHNYFYFFHML
... (len=241)

Organism: *Bos taurus*

HYGIRWGGILLLLYFHMGGYHHCNWHNYREGYVFYYMYNTITGGYCEIHWFGVWTWFCHCVMFYFGHYHH
... (len=245)

WLMWHFIFGFTKGLFGIHWGGLWHIHWYGCVIYIGIGYPELTPHYAGGGWWFPMHETEGHVYYYCEMYLH
... (len=248)

Organism: *Drosophila melanogaster*

ICCLNGHWWGGHYHHMMLGYYIPFDIFHSHMCIHHWYEGHGYMLYIWGYICLPHILMFAILNFHEIGH
... (len=245)

EFHESWWGILGGHYFYHWFLLHNTTPYWHIKLYYIWFIFHFHNHHGWLNGNWFLPGFYHFWPVICEYEI
... (len=240)

Organism: *Escherichia coli* (strain K12)

YHMCYHWFPGYCNPFYHMFYNHMWYHIEYMYMIWCYGMGGWLMYQIGDIGCYLFWLPLYGAIGIWKLL
... (len=251)

HHCFIHLPGWFLGYIYLHYHGFYLVHHIYLCHITGHQEKFGVYHVFHYPWHFIFYWFEFYIIIGINPGFG
... (len=245)

Organism: *Homo sapiens*

MGHIVFHFGYRWCLWGGGMWHHGYWREYNFEKYIHLGMYGFYHLLHYNIGYPGFHWVFCQFHFHFLDCE
... (len=242)

LHGHYLFNRMHYMYMYVHYYPHIFYMIHMIYLYCHGFHRGGWYKFLFGWFFHCFMFHGLRHVLGFWHYGY
... (len=246)

Organism: *Mus musculus*

VGWWQCHWCEFIHGCNKIWCLCHIAWWYMGTYLFYIMMGHVYYFYELFIHYFYIVPGQYHCWYYIYLF
... (len=249)

INHWRGPYFWYEFYFEWCCWEDIFYIHGIHGHTHYYWWSWWYLMGGEWWLEYGYDIYGCQGLLYNEHHEF
... (len=244)

Organism: *Oryza sativa* subsp. *japonica*

HFGHYMILIGYGYEHLNVFCFWPHYGYWHWFLYVLMHGYWFMICYGFFYYDMFHYYFPHLFCITWNMFYL
... (len=247)

```
GILGWIFMFCLQYLEHGIYCWGGYMGYRFVWFIWVIWMFYVFIMWGLILMILFEHHYGGEHYFMTSEIEM
... (len=246)
```

Organism: *Rattus norvegicus*

```
YFEYCTIWYHEGLMIHPEGIYLMHCIFHIYTEYWIFIWSFHGWVHDTFEFCWMIWYGGTHTIYYFHGY
... (len=246)
```

```
RPSFNLHHYYYGMIDHGHMGGLYWNEIHWQHMHIYYEIREWHIEPTNFSCWYGYGGWHFGEFCLMITRF
... (len=248)
```

Organism: *Saccharomyces cerevisiae* (strain ATCC 204508 / S288c)

```
NFEIILVHYHHIHIKMSGYIDMLLYIHHHEGGCNFLAWCHIDYYGIWGFLFMFLYHHHPWMYTPWIWG
... (len=241)
```

```
EFHIWYQGCPHYWIECEKYGYPHYQYYWHEGIYCFMIHCFHYHCRPYNHVFHIFEYVIELIWYYFFIFPQ
... (len=249)
```

Organism: *Xenopus laevis*

```
KMGGYYNLGECHMIGCHWYWFYYCYICMLWKIWFIEGFGPCMEYIQYHFHWNHYPFWNHMTLLLHAF
... (len=242)
```

```
HGHYPYWYGYHMHGPHLYYDHVGYFWHEHQYYHACHMHNMYGWFMIFGYMEHYLHWKGLYLWYWHYLC
... (len=239)
```

```
# — CELL 15: Mode Collapse + AA Frequency Analysis —————
all_gen = []
for cls in range(CGAN_N_CLASSES):
    all_gen.extend(G.generate_sequences(n=10, label_id=cls))
unique_ratio = len(set(all_gen)) / len(all_gen)
print(f'cGAN — Generated: {len(all_gen)} | Unique: {len(set(all_gen))}
| Ratio: {unique_ratio:.3f}')
if unique_ratio < 0.5: print('WARNING: Severe mode collapse!')
elif unique_ratio < 0.8: print('WARNING: Mild mode collapse.')
else: print('No severe mode collapse detected.')
```

```
# AA Frequency comparison
```

```
def aa_frequency(sequences):
    counts = {aa: 0 for aa in AA_LIST}
    total = 0
    for seq in sequences:
        for aa in seq:
            if aa in counts:
                counts[aa] += 1; total += 1
    return {aa: (c / total if total else 0) for aa, c in
counts.items()}
```

```
real_seqs = df['sequence'].sample(500, random_state=SEED).tolist()
```



```

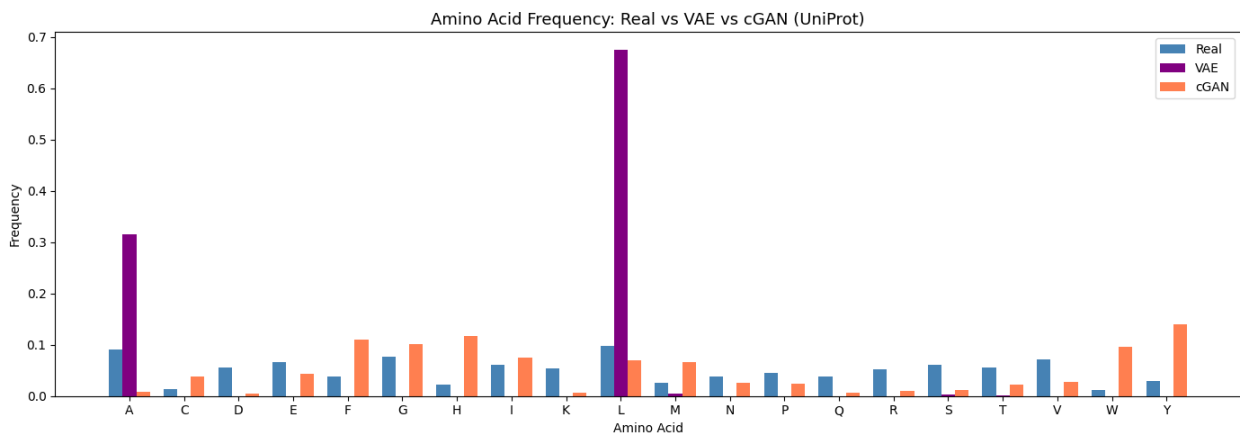
vae_seqs = vae_model.generate(n=200)
cgan_seqs = G.generate_sequences(n=200, label_id=0)

real_freq = aa_frequency(real_seqs)
vae_freq = aa_frequency(vae_seqs)
cgan_freq = aa_frequency(cgan_seqs)

x = np.arange(len(AA_LIST)); w = 0.25
fig, ax = plt.subplots(figsize=(14, 5))
ax.bar(x - w, [real_freq[a] for a in AA_LIST], w, label='Real',
color='steelblue')
ax.bar(x, [vae_freq[a] for a in AA_LIST], w, label='VAE',
color='purple')
ax.bar(x + w, [cgan_freq[a] for a in AA_LIST], w, label='cGAN',
color='coral')
ax.set_xticks(x); ax.set_xticklabels(AA_LIST)
ax.set_title('Amino Acid Frequency: Real vs VAE vs cGAN (UniProt)',
fontsize=13)
ax.set_xlabel('Amino Acid'); ax.set_ylabel('Frequency'); ax.legend()
plt.tight_layout()
plt.savefig('aa_freq_uniprot.png', dpi=150)
plt.show()

cGAN – Generated: 100 | Unique: 100 | Ratio: 1.000
No severe mode collapse detected.

```



```

# — CELL 16: Final Summary —————
print('='*70)
print('FINAL SUMMARY – DeepGen-Protein (UniProt)')
print('='*70)
print(f'Dataset   : UniProtKB/Swiss-Prot')
print(f'Sequences: {len(df):,} (after filtering)')
print()
print('ISA-2: Generative Models')
print(f'  {"Model":<10} {"Samples":>8} {"Unique":>8} {"AA Entropy":>12}')

```

```

print(' ' + '-'*42)
for name, seqs in [("VAE", vae_seqs), ("cGAN", cgan_seqs)]:
    freq = aa_frequency(seqs)
    entropy = -sum(p * np.log(p + 1e-9) for p in freq.values())
    print(f' {name:<10} {len(seqs):>8} {len(set(seqs)):>8}
{entropy:>12.3f}')
print()
print('Saved files:')
print(' vae_uniprot.pt | cgan_G_uniprot.pt | cgan_D_uniprot.pt')
print(' vae_uniprot_curves.png | vae_uniprot_tsne.png')
print(' cgan_uniprot_curves.png | aa_freq_uniprot.png')

```

FINAL SUMMARY – DeepGen-Protein (UniProt)

Dataset : UniProtKB/Swiss-Prot
Sequences: 39,576 (after filtering)

ISA-2: Generative Models

Model	Samples	Unique	AA Entropy

VAE	200	35	0.676
cGAN	200	200	2.633

Saved files:

vae_uniprot.pt | cgan_G_uniprot.pt | cgan_D_uniprot.pt
vae_uniprot_curves.png | vae_uniprot_tsne.png
cgan_uniprot_curves.png | aa_freq_uniprot.png